

HTML + CSS

DESARROLLO DE INTERFACES WEB

F.P. G.S. DESARROLLO APLICACIONES WEB



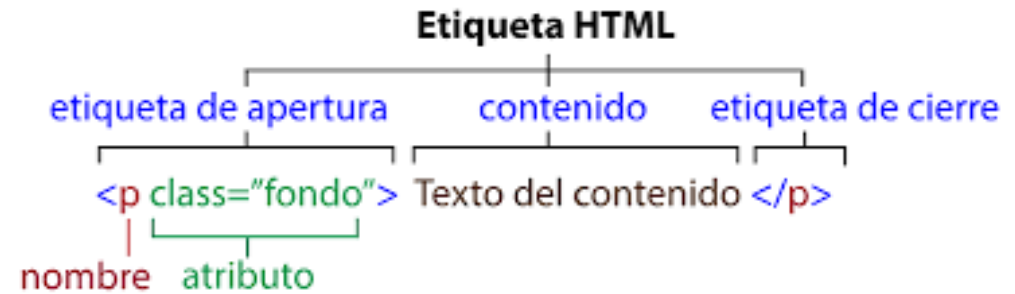
CEU FP

Formación Profesional
Madrid

HTML

Etiquetas en HTML

- ▶ a, abbr, address, area, article, aside, audio, b, base, bdi, bdo, blockquote, body, br, button, canvas, caption, cite, code, col, colgroup, data, datalist, dd, del, details, dfn, dialog, div, dl, dt, em, embed, fieldset, figcaption, figure, footer, form, h1, h2, h3, h4, h5, h6, head, header, hr, html, i, iframe, img, input, ins, kbd, label, legend, li, link, main, map, mark, meta, meter, nav, noscript, object, ol, optgroup, option, output, p, param, picture, pre, progress, q, rp, rt, ruby, s, samp, script, section, select, small, source, span, strong, style, sub, summary, sup, table, tbody, td, template, textarea, tfoot, th, thead, time, title, tr, track, u, ul, var, video, wbr



Comentarios <!-- ... -->

Etiquetas que dan formato

- ▶ **<code>**: Representa un fragmento de código, mostrando texto en una fuente monoespaciada para distinguirlo del texto normal.
- ▶ **<cite>**: Representa el título de una obra citada, como un libro, artículo o película, y suele mostrarse en cursiva.
- ▶ ****: Indica texto que ha sido eliminado del documento, generalmente mostrado con una línea que lo atraviesa (tachado).
- ▶ **<ins>**: Indica texto que ha sido insertado en el documento, a menudo mostrado subrayado para resaltar su adición.

`<p>Mi color favorito es el azul rojo!</p>`

Mi color favorito es el azul rojo!

Etiquetas que dan formato

- **<meter>**: Muestra una medida escalar dentro de un rango conocido, como una calificación o un nivel, mediante una barra de medidor.

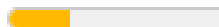
```
<p><label for="anna">Anna's score:</label>  
<meter id="anna" min="0" low="40" high="90" max="100" value="95"></meter></p>
```

Anna's score: 

```
<p><label for="peter">Peter's score:</label>  
<meter id="peter" min="0" low="40" high="90" max="100" value="65"></meter></p>
```

Peter's score: 

```
<p><label for="linda">Linda's score:</label>  
<meter id="linda" min="10" low="40" high="90" max="100" value="35"></meter></p>
```

Linda's score: 

Etiquetas que dan formato

- ▶ **<progress>**: Representa el progreso de una tarea en ejecución, mostrando visualmente el porcentaje completado mediante una barra de progreso.

```
<label for="file">Downloading progress:</label>  
<progress id="file" value="32" max="100"> 32% </progress>
```

Downloading progress: 

Formularios

- ▶ **<form>**: Crea un contenedor para un formulario que recopila entradas del usuario y permite enviar datos a un servidor. Atributos:
 - ▶ **action**: Define la URL de destino para los datos del formulario.
 - ▶ **method**: Especifica el método HTTP ("get" o "post") para el envío.
 - ▶ **enctype**: Indica cómo se codifican los datos (especialmente al subir archivos).
 - ▶ **name**: Asigna un identificador al formulario.
 - ▶ **target**: Determina dónde se muestra la respuesta.
 - ▶ **autocomplete**: Controla el autocompletado del navegador.
 - ▶ **novalidate**: Desactiva la validación HTML5 del navegador.
 - ▶ **accept-charset**: Establece el conjunto de caracteres aceptado.
 - ▶ **rel**: Mejora la seguridad al abrir enlaces en nuevas ventanas.
 - ▶ **onsubmit**: Permite validar o manipular el formulario antes del envío.

Formularios

- ▶ **<form>**: Crea un contenedor para un formulario que recopila entradas del usuario y permite enviar datos a un servidor. Atributos:
 - ▶ **method**: Especifica el método HTTP ("get" o "post") para el envío.
 - ▶ **"get"**: Envía los datos anexados a la URL. Adecuado para solicitudes que no modifican datos en el servidor.
 - ▶ **"post"**: Envía los datos en el cuerpo de la solicitud HTTP. Útil para datos sensibles o grandes cantidades de información.
 - ▶ **enctype**: Indica cómo se codifican los datos (especialmente al subir archivos).
 - ▶ **"application/x-www-form-urlencoded"**: Valor por defecto. Codifica los datos como pares clave-valor.
 - ▶ **"multipart/form-data"**: Se utiliza al enviar archivos. Permite subir archivos binarios.
 - ▶ **"text/plain"**: Envía datos sin codificar (poco común).

Formularios

- ▶ **<form>**: Crea un contenedor para un formulario que recopila entradas del usuario y permite enviar datos a un servidor. Atributos:
 - ▶ **target**: Determina dónde se muestra la respuesta.
 - ▶ **"_self"**: (Por defecto) La respuesta se carga en la misma ventana.
 - ▶ **"_blank"**: La respuesta se abre en una nueva ventana o pestaña.
 - ▶ **"_parent"**: La respuesta se carga en el marco padre.
 - ▶ **"_top"**: La respuesta se carga en la ventana completa.
 - ▶ **"nombre_del_frame"**: Carga la respuesta en un frame específico.
 - ▶ **rel**: Mejora la seguridad al abrir enlaces en nuevas ventanas.
 - ▶ Cuando se utiliza **target="_blank"** añadir **rel="noopener noreferrer"**

Formularios

- ▶ **<input>**: Crea un control interactivo en un formulario, como campos de texto, botones, casillas de verificación, etc., utilizando el atributo `type` para especificar su función.
- ▶ **<label>**: Proporciona una etiqueta descriptiva para un elemento de formulario, mejorando la accesibilidad al asociar texto con un control específico.
- ▶ **<select>**: Crea un menú desplegable que permite al usuario seleccionar una opción de una lista predefinida.
- ▶ **<option>**: Define una opción dentro de un elemento `<select>`, `<optgroup>` o `<datalist>`.
- ▶ **<textarea>**: Crea un campo de entrada de texto de varias líneas, permitiendo al usuario ingresar párrafos completos.

Formularios

- ▶ **<button>**: Crea un botón clicable que puede activar acciones al ser pulsado, como enviar un formulario o ejecutar scripts.
- ▶ **<fieldset>**: Agrupa elementos relacionados dentro de un formulario, generalmente con un borde alrededor, para organizar visualmente los controles.
- ▶ **<legend>**: Proporciona un título o leyenda para un elemento `<fieldset>`, describiendo el grupo de controles que contiene.
- ▶ **<datalist>**: Proporciona una lista de opciones predefinidas para un elemento `<input>`, permitiendo sugerencias automáticas mientras el usuario escribe.
- ▶ **<optgroup>**: Agrupa opciones relacionadas dentro de un `<select>`, organizando las opciones en categorías bajo un mismo encabezado.
- ▶ **<output>**: Representa el resultado de un cálculo o acción del usuario, mostrando información dinámica dentro de un formulario.

Formularios: Etiqueta input

- ▶ **<input>**: Crea un control interactivo en un formulario, como campos de texto, botones, casillas de verificación, etc., utilizando el atributo type para especificar su función.
 - ▶ **type="text"**: Crea un campo de entrada de texto de una sola línea.
 - ▶ **type="password"**: Genera un campo donde los caracteres ingresados se ocultan, ideal para contraseñas.
 - ▶ **type="email"**: Campo de entrada diseñado para direcciones de correo electrónico, validando el formato correcto.
 - ▶ **type="number"**: Campo que acepta solo números, a menudo con flechas para incrementar o disminuir el valor.
 - ▶ **type="checkbox"**: Crea una casilla que puede ser marcada o desmarcada, permitiendo selecciones múltiples.
 - ▶ **type="radio"**: Crea botones de opción donde solo una opción dentro del mismo grupo puede ser seleccionada.
 - ▶ **type="submit"**: Crea un botón que envía el formulario al servidor cuando es pulsado.
 - ▶ **type="reset"**: Genera un botón que reinicia los campos del formulario a sus valores iniciales.

Formularios: Etiqueta input

- ▶ **<input>**: Crea un control interactivo en un formulario, como campos de texto, botones, casillas de verificación, etc., utilizando el atributo type para especificar su función.
 - ▶ **type="button"**: Crea un botón estándar que puede asociarse a eventos personalizados mediante scripts.
 - ▶ **type="date"**: Proporciona un selector de fecha que permite al usuario elegir una fecha de un calendario.
 - ▶ **type="color"**: Muestra un selector que permite al usuario elegir un color.
 - ▶ **type="range"**: Crea un control deslizante para seleccionar un valor dentro de un rango especificado.
 - ▶ **type="search"**: Campo de entrada optimizado para búsquedas, con características adicionales en algunos navegadores.
 - ▶ **type="tel"**: Campo de entrada para números de teléfono, mostrando un teclado numérico en dispositivos móviles.
 - ▶ **type="url"**: Campo de entrada para URLs, validando que el formato sea una dirección web correcta.
 - ▶ **type="file"**: Permite al usuario seleccionar archivos de su sistema para subirlos al servidor.

Ejemplo 1

ⓘ http://127.0.0.1:5500/Ejemplos/Ejemplo1.html

Formulario con Todos los Tipos de Input

Información Personal (fieldset y legend)

Texto ():

Contraseña ():

Correo Electrónico ():

Número ():

Fecha ():

Fecha y Hora Local ():

Mes y Año ():

Semana y Año ():

Hora ():

Color Favorito ():

Sitio Web ():

Teléfono ():

Buscar ():

Rango (0-100) ():

Subir Archivo (): No file chosen

Intereses (☐):

- ☐ Deporte
- ☐ Música
- ☐ Lectura

Género (☐):

- ☐ Hombre
- ☐ Mujer
- ☐ Otro

Enviar con Imagen (): 

Botón ():

Ejercicio 1

- ▶ Crea una página HTML que contenga un formulario de registro de usuario. El formulario debe incluir los siguientes campos:
 - **Nombre completo** (input type="text") *(requerido)*.
 - **Correo electrónico** (input type="email") *(requerido)*.
 - **Contraseña** (input type="password") *(requerido)*.
 - **Fecha de nacimiento** (input type="date").
 - **Género** (input type="radio") con opciones "Masculino" y "Femenino".
 - **Intereses** (input type="checkbox") con opciones "Deportes", "Música", "Tecnología".
 - **País** (select con option) que incluya al menos tres países.
 - **Foto de perfil** (input type="file").
 - **Un botón para enviar** el formulario (input type="submit").
 - **Un botón para restablecer** el formulario (input type="reset").
- ▶ El formulario debe enviar los datos a "registro.php" utilizando el método POST y debe tener validación HTML5 donde sea apropiado. Utiliza etiquetas <label> para cada campo y asegúrate de que el formulario sea accesible.

Ejercicio 2

- ▶ Desarrolla una página HTML que contenga un formulario de contacto. El formulario debe incluir:
 - **Nombre** (input type="text") (requerido).
 - **Correo electrónico** (input type="email") (requerido).
 - **Teléfono** (input type="tel") con un patrón que acepte 9 dígitos.
 - **Asunto** (select con opciones "Consulta", "Sugerencia", "Reclamo").
 - **Mensaje** (textarea) (requerido).
 - **Un campo oculto** (input type="hidden") que incluya un token de seguridad.
 - **Un botón para enviar** el formulario (button type="submit").
 - **Un botón para restablecer** el formulario (button type="reset").
- ▶ El formulario debe enviar los datos a "contacto.php" utilizando el método POST. Implementa validación HTML5 y utiliza atributos pattern y required donde corresponda. Usa etiquetas <label> y agrupa los campos relacionados con <fieldset> y <legend>.

Ejercicio 3:

- ▶ Crea una página HTML que contenga un formulario para una encuesta de satisfacción. El formulario debe incluir:
 - **Nombre** (input type="text") (*requerido*).
 - **Edad** (input type="number") entre 18 y 100 (*requerido*).
 - **Fecha de visita** (input type="date") (*requerido*).
 - **Calificación general** (input type="range") de 1 a 10, mostrando el valor seleccionado en un elemento <output>.
 - **Servicios utilizados** (input type="checkbox") con opciones "Restaurante", "Spa", "Gimnasio", "Piscina".
 - **Comentarios adicionales** (textarea).
 - **Un campo para seleccionar el color favorito** (input type="color").
 - **Un botón para enviar** el formulario (input type="submit").
 - **Un botón para restablecer** el formulario (input type="reset").
- ▶ El formulario debe enviar los datos a "encuesta.php" utilizando el método POST y debe incluir validación HTML5. Utiliza etiquetas <label> y elementos <fieldset> y <legend> para organizar el formulario. Usa JavaScript para actualizar dinámicamente el valor del <output> cuando se modifica el rango.

Frames

- ▶ **<iframe>**: se utiliza para incrustar un documento HTML dentro de otro documento HTML.
 - ▶ **src**: Especifica la URL del documento que se va a incrustar.
 - ▶ **width y height**: Definen el ancho y alto del iframe en píxeles o porcentajes.
 - ▶ **title**: Proporciona una descripción del contenido para mejorar la accesibilidad.
 - ▶ **sandbox**: Aplica restricciones de seguridad al contenido incrustado.
 - ▶ **allow**: Especifica permisos para funciones como reproducción automática, pantalla completa, etc.
 - ▶ **loading**: Controla la carga diferida del iframe ("lazy" para carga diferida, "eager" para carga inmediata).

Imágenes

- ▶ ****: Inserta una imagen en la página web, especificando su fuente mediante el atributo src.
- ▶ **<map>**: Define un mapa de imagen del lado del cliente, permitiendo crear áreas interactivas dentro de una imagen.
 - ▶ **<area>**: Especifica una región dentro de un mapa de imagen, definiendo áreas clicables con formas y coordenadas específicas.
- ▶ **<canvas>**: Proporciona una superficie de dibujo en la que se pueden renderizar gráficos dinámicos mediante scripts, generalmente usando JavaScript.
- ▶ **<figcaption>**: Proporciona una leyenda o título para un elemento <figure>, describiendo el contenido visual o ilustrativo.
- ▶ **<figure>**: Representa contenido autónomo y auto-contenido, como una imagen, gráfico o código, que es referenciado en el texto principal pero puede moverse sin afectar el flujo.
- ▶ **<picture>**: Sirve como contenedor para múltiples recursos de imagen, permitiendo al navegador seleccionar la más adecuada según el contexto o las condiciones (como la resolución de pantalla).
- ▶ **<svg>**: Embebe contenido de gráficos vectoriales escalables (SVG) directamente en el documento, permitiendo la inclusión de imágenes vectoriales y animaciones.

Imágenes

- ▶ **<map>**: Define un mapa de imagen del lado del cliente, permitiendo crear áreas interactivas dentro de una imagen.
- ▶ **<area>**: Especifica una región dentro de un mapa de imagen, definiendo áreas clicables con formas y coordenadas específicas.
 - ▶ **shape**: Forma del área (rect, circle, poly).
 - ▶ **coords**: Coordenadas que definen el área.
 - ▶ **href**: Enlace al que se dirige cuando se hace clic.
 - ▶ **alt**: Texto alternativo para accesibilidad.

Imágenes

- ▶ **<canvas>**: Proporciona una superficie de dibujo en la que se pueden renderizar gráficos dinámicos mediante scripts, generalmente usando JavaScript.
- ▶ **<svg>**: Embebe contenido de gráficos vectoriales escalables (SVG) directamente en el documento, permitiendo la inclusión de imágenes vectoriales y animaciones.
 - ▶ **<circle>**: Dibuja un círculo.
 - ▶ **cx, cy**: Coordenadas del centro.
 - ▶ **r**: Radio.
 - ▶ **stroke**: Color del borde.
 - ▶ **stroke-width**: Grosor del borde.
 - ▶ **fill**: Color de relleno.

```
var canvas = document.getElementById('miCanvas');
var ctx = canvas.getContext('2d');

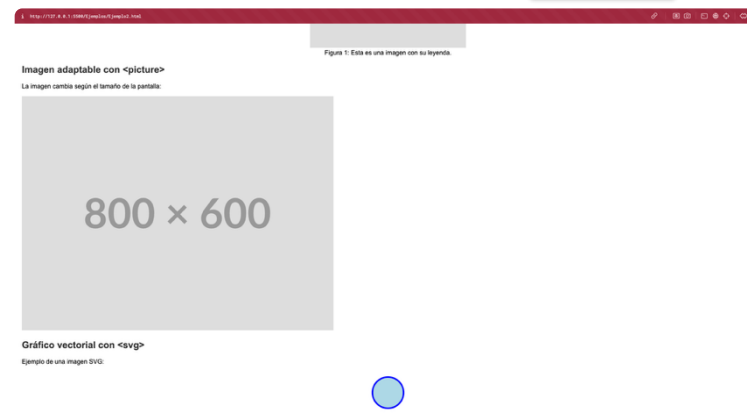
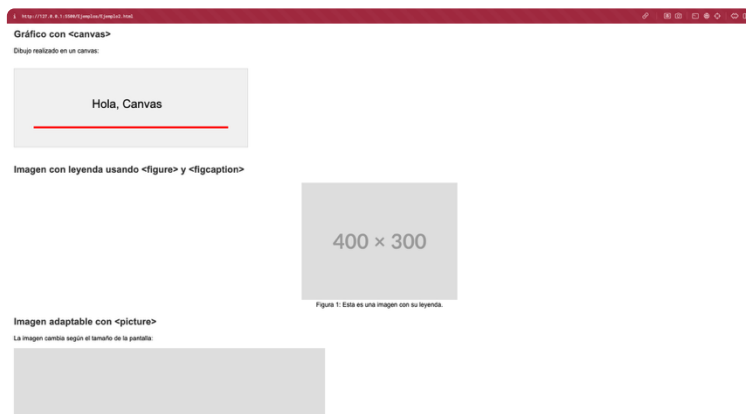
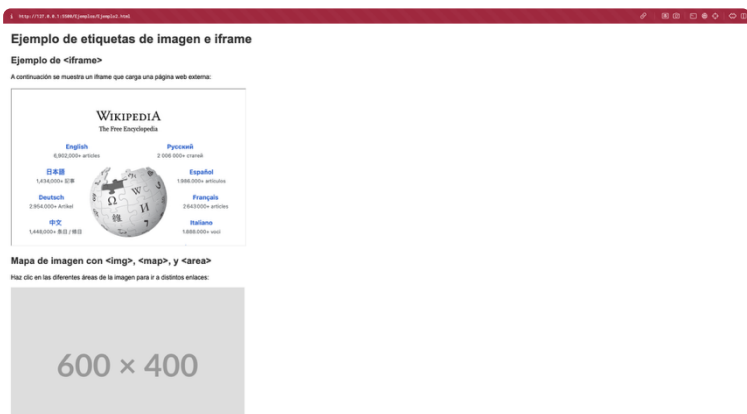
// Fondo
ctx.fillStyle = '#f0f0f0';
ctx.fillRect(0, 0, canvas.width, canvas.height);

// Texto
ctx.fillStyle = 'black';
ctx.font = '30px Arial';
ctx.fillText('Hola, Canvas', 200, 100);

// Línea
ctx.beginPath();
ctx.moveTo(50, 150);
ctx.lineTo(550, 150);
ctx.strokeStyle = 'red';
ctx.lineWidth = 5;
ctx.stroke();
```

Imágenes

- ▶ **<picture>**: Sirve como contenedor para múltiples recursos de imagen, permitiendo al navegador seleccionar la más adecuada según el contexto o las condiciones (como la resolución de pantalla).
 - ▶ **<source>**: Define recursos alternativos para diferentes medios o condiciones.
 - ▶ **media**: Especifica las condiciones (como ancho mínimo).
 - ▶ **srcset**: URL de la imagen que se usará si se cumple la condición.



Ejemplo 2

Ejercicio 4

- ▶ Crea una página HTML que contenga una imagen de 600x400 píxeles que represente un mapa o esquema. Utiliza las etiquetas `<map>` y `<area>` para definir tres áreas clicables dentro de la imagen:
 - ▶ Área 1: Cuando se haga clic, debe llevar al usuario a `https://www.example.com/area1`.
 - ▶ Área 2: Cuando se haga clic, debe llevar al usuario a `https://www.example.com/area2`.
 - ▶ Área 3: Cuando se haga clic, debe mostrar un mensaje emergente (alert) con el texto "Has hecho clic en el Área 3".
 - ▶ Usar function `mensajeArea3() { alert("Has hecho clic en el Área 3"); }`

Ejercicio 5

- ▶ Diseña una página HTML que:
 - ▶ Contenga un `<iframe>` que muestre la página principal de Wikipedia (<https://www.wikipedia.org>). El iframe debe tener un tamaño de 600x400 píxeles.
 - ▶ Debajo del iframe, incluye un gráfico vectorial simple utilizando la etiqueta `<svg>`. Dibuja un círculo de color azul con borde negro.
 - ▶ Extra: añade un svg como este (



Ejercicio 6

- ▶ Crea una página HTML que muestre una imagen que se adapte al tamaño de la pantalla utilizando la etiqueta `<picture>`. Debes proporcionar diferentes versiones de la imagen para distintas resoluciones:
 - ▶ Si el ancho de la pantalla es de 800 píxeles o más, muestra una imagen de 800x600 píxeles.
 - ▶ Si el ancho de la pantalla es de entre 500 y 799 píxeles, muestra una imagen de 500x375 píxeles.
 - ▶ Para pantallas menores de 500 píxeles de ancho, muestra una imagen de 300x225 píxeles.
 - ▶ Si el navegador no soporta la etiqueta `<picture>`, utiliza un `<img>` de respaldo con una imagen de 600x450 píxeles.
- ▶ Utiliza imágenes de relleno (placeholders) para este ejercicio. Asegúrate de incluir el atributo `alt` para la accesibilidad.

Audio y video

- ▶ **<audio>**: Inserta contenido de audio en la página web y permite controlarlo (reproducir, pausar, etc.).
- ▶ **<source>**: Especifica múltiples recursos de medios para elementos como <video>, <audio> y <picture>, permitiendo al navegador elegir el más adecuado.
- ▶ **<track>**: Proporciona pistas de texto (subtítulos, descripciones) para elementos multimedia como <video> y <audio>.
- ▶ **<video>**: Inserta contenido de video en la página web y permite controlarlo (reproducir, pausar, pantalla completa, etc.).

Audio y video

- ▶ **<audio>**: Inserta contenido de audio en la página web y permite controlarlo (reproducir, pausar, etc.).
 - ▶ **src**: URL del archivo de audio (opcional si se usa <source>).
 - ▶ **controls**: Muestra controles de reproducción como play, pause y volumen.
 - ▶ **autoplay**: Reproduce automáticamente el audio al cargar la página.
 - ▶ **loop**: Hace que el audio se repita indefinidamente.
 - ▶ **muted**: Silencia el audio por defecto.
 - ▶ **preload**: Sugiere al navegador cómo manejar el archivo antes de la reproducción (none, metadata, auto).

Audio y video

- ▶ **<source>**: Especifica múltiples recursos de medios para elementos como `<video>`, `<audio>` y `<picture>`, permitiendo al navegador elegir el más adecuado.
 - ▶ **src**: URL del archivo de medios.
 - ▶ **type**: Tipo de archivo multimedia (por ejemplo, `audio/mpeg`, `video/mp4`).
 - ▶ **media**: Especifica condiciones para usar este recurso (consulta de medios, como en CSS).

Audio y video

- ▶ **<track>**: Proporciona pistas de texto (subtítulos, descripciones) para elementos multimedia <video> (también <audio>)
 - ▶ **src**: URL del archivo de pista (por ejemplo, un archivo .vtt).
 - ▶ **kind**: Especifica el tipo de pista (subtitles, captions, descriptions, chapters, metadata).
 - ▶ **label**: Proporciona un nombre para la pista, mostrado en los controles del reproductor.
 - ▶ **srclang**: Especifica el idioma de la pista (por ejemplo, en, es).
 - ▶ **default**: Indica que esta pista debe mostrarse por defecto.

Audio y video

- ▶ **<video>**: Inserta contenido de video en la página web y permite controlarlo (reproducir, pausar, pantalla completa, etc.).
 - ▶ **src**: URL del archivo de video (opcional si se usa <source>).
 - ▶ **width**: Ancho del video (en píxeles).
 - ▶ **height**: Altura del video (en píxeles).
 - ▶ **controls**: Muestra controles de reproducción como play, pause, volumen y pantalla completa.
 - ▶ **autoplay**: Reproduce automáticamente el video al cargar la página.
 - ▶ **loop**: Hace que el video se repita indefinidamente.
 - ▶ **muted**: Silencia el video por defecto.
 - ▶ **poster**: URL de una imagen que se muestra mientras se carga el video o antes de que comience.
 - ▶ **preload**: Sugiere al navegador cómo manejar el archivo antes de la reproducción (none, metadata, auto).

Ejemplo de etiqueta <audio>



Ejemplo de etiqueta <video>



Ejemplo 3

Ejercicio 7

- ▶ Desarrolla una página web en HTML que cumpla con los siguientes requisitos utilizando las etiquetas `<audio>`, `<video>`, `<source>` y `<track>`:
- ▶ Reproductor de Audio:
 - ▶ Inserta un reproductor de audio que permita al usuario reproducir, pausar y controlar el volumen.
 - ▶ Proporciona al menos dos formatos de audio diferentes (por ejemplo, MP3 y OGG) utilizando la etiqueta `<source>` para asegurar la compatibilidad con distintos navegadores.
 - ▶ Añade un mensaje alternativo que se mostrará si el navegador no soporta el elemento `<audio>`.
- ▶ Reproductor de Video:
 - ▶ Inserta un reproductor de video con controles de reproducción, incluyendo opción de pantalla completa.
 - ▶ Proporciona al menos dos formatos de video diferentes (por ejemplo, MP4 y WebM) utilizando la etiqueta `<source>`.
 - ▶ Añade subtítulos en español al video utilizando la etiqueta `<track>` y un archivo de subtítulos en formato WebVTT.
 - ▶ Incluye un mensaje alternativo que se mostrará si el navegador no soporta el elemento `<video>`.

Listas

- ▶ **<menu>**: Define una lista alternativa no ordenada, utilizada principalmente para menús contextuales o de herramientas.
- ▶ ****: Define una lista no ordenada donde los elementos están marcados con viñetas.
- ▶ ****: Define una lista ordenada donde los elementos están numerados.
 - ▶ **type**: Define el estilo de numeración (1, A, a, I, i).
 - ▶ **start**: Especifica el número inicial de la lista.
 - ▶ **reversed**: Invierte el orden de la numeración.
- ▶ ****: Define un elemento de lista dentro de , , <menu> o <dir>.
- ▶ **<dl>**: Define una lista de descripciones, útil para pares de términos y definiciones.
 - ▶ **<dt>**: Define un término o nombre dentro de una lista de descripciones <dl>.
 - ▶ **<dd>**: Define la descripción o definición de un término en una lista <dl>.

Tablas

- ▶ **<table>**: Define una tabla en un documento HTML para organizar datos en filas y columnas.
 - ▶ **<caption>**: Especifica un título o leyenda para una tabla.
 - ▶ **<th>**: Define una celda de encabezado en una tabla, generalmente se muestra en negrita y centrada.
 - ▶ **<tr>**: Define una fila en una tabla que contiene celdas de datos o encabezados.
 - ▶ **<td>**: Define una celda de datos estándar en una tabla.
 - ▶ **<thead>**: Agrupa el contenido del encabezado (filas de <th>) en una tabla.
 - ▶ **<tbody>**: Agrupa el contenido del cuerpo (filas de <td>) en una tabla.
 - ▶ **<tfoot>**: Agrupa el contenido del pie de página en una tabla, útil para resúmenes o totales.
 - ▶ **<col>**: Especifica propiedades de estilo para una columna dentro de un <colgroup>.
 - ▶ **<colgroup>**: Especifica un grupo de una o más columnas en una tabla para aplicar estilos.

Tablas

- ▶ **<table>**: Define una tabla en un documento HTML para organizar datos en filas y columnas.
 - ▶ **<th>**: Define una celda de encabezado en una tabla, generalmente se muestra en negrita y centrada.
 - ▶ **colspan**: Número de columnas que abarca la celda de encabezado.
 - ▶ **rowspan**: Número de filas que abarca la celda de encabezado.
 - ▶ **headers**: Referencia a los id de las celdas de encabezado asociadas.
 - ▶ **scope**: Define el alcance de la celda de encabezado (col, row, colgroup, rowgroup).
 - ▶ **<td>**: Define una celda de datos estándar en una tabla.
 - ▶ **colspan**: Número de columnas que abarca la celda.
 - ▶ **rowspan**: Número de filas que abarca la celda.
 - ▶ **headers**: Referencia a los id de las celdas de encabezado asociadas.
 - ▶ **<col>**: Especifica propiedades de estilo para una columna dentro de un <colgroup>.
 - ▶ **span**: Especifica el número de columnas a las que se aplican las propiedades del <col>.
 - ▶ **<colgroup>**: Especifica un grupo de una o más columnas en una tabla para aplicar estilos.
 - ▶ **span**: Especifica el número de columnas en el grupo.

Ejemplos de Listas

1. Lista no ordenada ()

- <i> Elemento A </i>
- <i> Elemento B </i>
- <i> Elemento C </i>

2. Lista ordenada ()

- <i> Primer elemento </i>
- <i> Segundo elemento </i>
- <i> Tercer elemento </i>

3. Lista de descripción (<dl>)

<d> HTML </d>
 <dd> Lenguaje de marcado para la creación de páginas web. </dd>
<d> CSS </d>
 <dd> Lenguaje de estilos para diseñar páginas web. </dd>
<d> JavaScript </d>
 <dd> Lenguaje de programación para agregar interactividad a las páginas web. </dd>

4. Menú (<menu>)

- <i> Inicio </i>
- <i> Servicios </i>
- <i> Contacto </i>

5. Lista de directorio (<dir>)

Nota: La etiqueta <dir> está obsoleta en HTML5; se muestra aquí solo con fines ilustrativos. Usa en su lugar.

- <i> Archivo1.txt </i>
- <i> Imagen.png </i>
- <i> Documento.pdf </i>

Ejemplos de Tablas

<caption> Tabla de Productos </caption>		
<th> Producto </th>	<th> Precio </th>	<th> Cantidad </th>
<td> Manzanas </td>	<td> \$1.00 </td>	<td> 50 </td>
<td> Naranjas </td>	<td> \$1.20 </td>	<td> 75 </td>
<td> Peras </td>	<td> \$1.50 </td>	<td> 60 </td>
<td colspan="2"> Total de Productos </td>		<td> 185 </td>

Ejercicio 8

- ▶ Crea una página web en HTML que cumpla con los siguientes requisitos:
 - ▶ Diseña una tabla que muestre información sobre tres cursos diferentes. La tabla debe contener las siguientes columnas:
 - ▶ Nombre del curso
 - ▶ Duración
 - ▶ Detalles
 - ▶ En la columna Detalles, para cada curso, incluye una lista de descripción que contenga al menos dos elementos:
 - ▶ Instructor: Nombre del instructor del curso.
 - ▶ Nivel: Nivel de dificultad del curso (por ejemplo, Principiante, Intermedio, Avanzado).
 - ▶ Utiliza las etiquetas de tabla y lista apropiadas para estructurar el contenido, específicamente:
 - ▶ Etiquetas de tabla: `<table>`, `<caption>`, `<thead>`, `<tbody>`, `<tr>`, `<th>`, `<td>`
 - ▶ Etiquetas de lista de descripción: `<dl>`, `<dt>`, `<dd>`

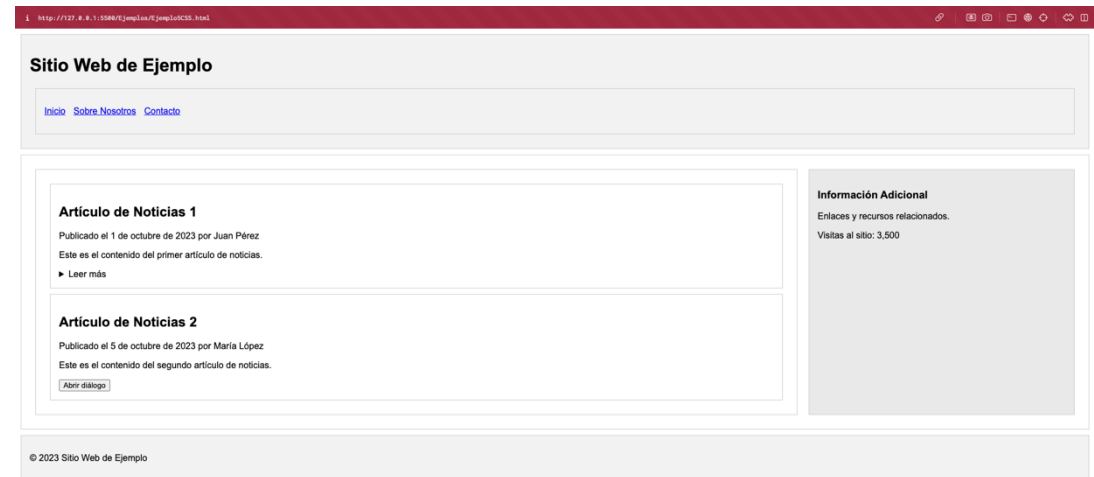
Layout

- ▶ `<div>`: Define un contenedor genérico en un documento, utilizado para agrupar bloques de contenido y aplicar estilos o scripts.
- ▶ `<span>`: Define un contenedor en línea para agrupar contenido y aplicar estilos o scripts sin interrumpir el flujo del texto.
- ▶ `<header>`: Define un encabezado para un documento o sección, que puede contener títulos, logotipos o elementos de navegación.
- ▶ `<footer>`: Define un pie de página para un documento o sección, que puede contener información de autor, enlaces adicionales o notas legales.
- ▶ `<main>`: Especifica el contenido principal y único de un documento, excluyendo encabezados, pies de página y barras laterales.
 - ▶ Debe haber solo un `<main>` por página y no debe estar anidado dentro de `<article>`, `<aside>`, `<footer>`, `<header>`, o `<nav>`.
- ▶ `<section>`: Define una sección temática en un documento, utilizada para agrupar contenido relacionado.

Layout

- ▶ `<article>`: Define contenido independiente y auto-contenido, como artículos, blogs o publicaciones de noticias.
- ▶ `<aside>`: Define contenido aparte del contenido principal, como barras laterales, notas al margen o publicidad.
- ▶ `<details>`: Define detalles adicionales que el usuario puede mostrar u ocultar, creando un elemento desplegable.
- ▶ `<dialog>`: Define un cuadro de diálogo o ventana emergente, utilizado para modales o interacciones con el usuario.
- ▶ `<summary>`: Define un encabezado visible para un elemento `<details>`, que al hacer clic muestra u oculta el contenido.
- ▶ `<data>`: Proporciona una traducción legible por máquina de un contenido dado, útil para datos específicos como fechas o medidas.

Ejemplo 5



Ejercicio 9

- ▶ Crea una página web en HTML para una receta de cocina. La página debe seguir estos requisitos:
- ▶ Utiliza las etiquetas `<header>`, `<main>`, `<article>`, `<section>`, `<aside>`, `<footer>` para estructurar el documento.
- ▶ En el `<header>`, incluye el nombre del sitio (por ejemplo, "Recetas Deliciosas") y un menú de navegación con enlaces a otras recetas (pueden ser enlaces ficticios).
- ▶ En `<main>`, utiliza un `<article>` para la receta, que debe incluir:
 - ▶ Un título (`<h1>`) con el nombre de la receta.
 - ▶ Una imagen de la receta (puede ser un espacio reservado).
 - ▶ Una sección `<section>` para los ingredientes, utilizando una lista.
 - ▶ Una sección `<section>` para los pasos de preparación, numerados en una lista ordenada.
 - ▶ Un `<details>` con sugerencias adicionales o consejos, con un `<summary>` que diga "Consejos adicionales".
- ▶ Añade un `<aside>` que contenga información nutricional, utilizando la etiqueta `<data>` para especificar valores numéricos (por ejemplo, calorías, proteínas, grasas).
- ▶ En el `<footer>`, incluye información sobre derechos de autor y un enlace a una página de contacto.

Atributos globales HTML

- ▶ Los **atributos globales** en HTML son atributos que pueden aplicarse a cualquier elemento HTML, independientemente de su tipo. Estos atributos permiten añadir información adicional, controlar el comportamiento y aplicar estilos a los elementos de la página, mejorando así su funcionalidad y accesibilidad.

Atributos globales HTML

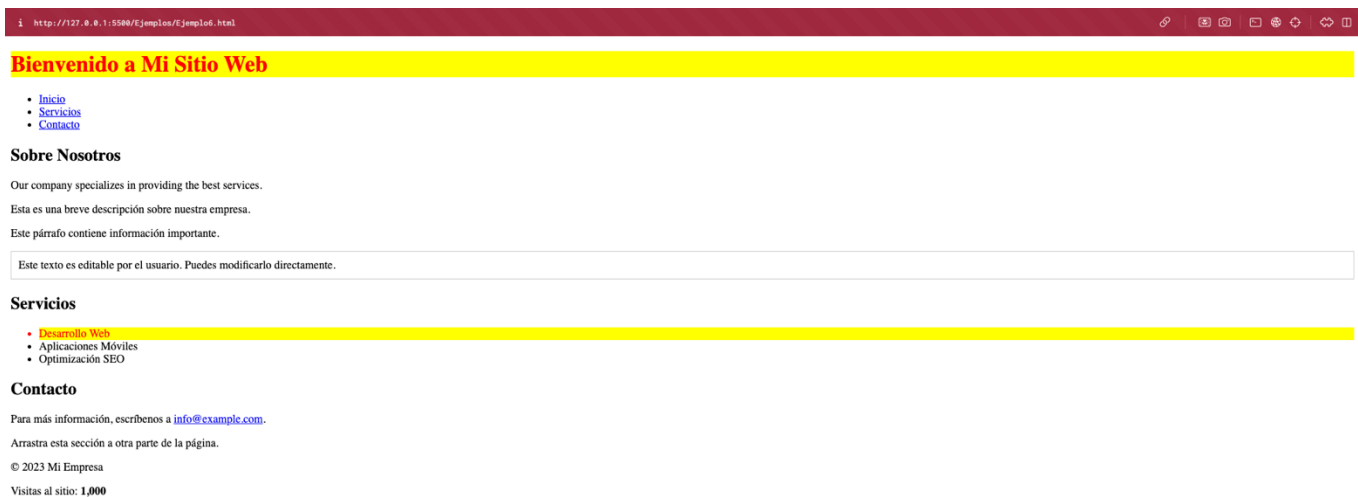
- ▶ **id:** Especifica un identificador único para el elemento, que puede ser utilizado por CSS y JavaScript para manipular o referenciar ese elemento específico.
- ▶ **class:** Asigna uno o varios nombres de clase al elemento, permitiendo aplicar estilos CSS comunes o seleccionar el elemento mediante scripts.
- ▶ **style:** Permite aplicar estilos CSS en línea directamente al elemento, modificando su apariencia.
- ▶ **title:** Proporciona información adicional sobre el elemento, que generalmente se muestra como un tooltip cuando el usuario pasa el cursor sobre él.
- ▶ **lang:** Especifica el idioma del contenido del elemento, ayudando en la internacionalización y en la correcta pronunciación por lectores de pantalla.
- ▶ **dir:** Indica la dirección del texto del elemento, ya sea de izquierda a derecha (ltr) o de derecha a izquierda (rtl), útil para idiomas como el árabe o hebreo.
- ▶ **data-*:** Permite almacenar datos personalizados en el elemento, donde * puede ser cualquier nombre; estos datos pueden ser accedidos mediante JavaScript.

Atributos globales HTML

- ▶ **hidden:** Indica que el elemento no es relevante y no se muestra al usuario, aunque permanece en el DOM.
- ▶ **tabindex:** Define el orden en que los elementos reciben el foco al navegar con la tecla Tab, mejorando la navegación por teclado.
- ▶ **accesskey:** Asigna una tecla o combinación de teclas de acceso rápido para enfocar o activar el elemento.
- ▶ **draggable:** Especifica si el elemento puede ser arrastrado (true o false) en operaciones de arrastrar y soltar.
- ▶ **spellcheck:** Indica si el elemento debe ser revisado ortográficamente (true o false), útil en campos de entrada de texto.
- ▶ **contenteditable:** Indica si el contenido del elemento es editable por el usuario (true o false), permitiendo modificar el texto en línea.
- ▶ **translate:** Indica si el contenido del elemento debe ser traducido cuando se utiliza un traductor automático (yes o no).

Atributos globales HTML

- ▶ **autocapitalize**: Controla la capitalización automática del texto en dispositivos móviles, con valores como none, sentences, words, characters.
- ▶ **inputmode**: Sugerencia para dispositivos con pantalla táctil sobre el tipo de teclado a mostrar, como text, numeric, decimal, tel, email, url.
- ▶ **is**: Utilizado en Web Components, especifica el tipo de un elemento personalizado que extiende de un elemento estándar.
- ▶ **itemprop, itemtype, itemscope, itemid, itemref**: Atributos relacionados con microdatos que permiten marcar semánticamente el contenido para motores de búsqueda y aplicaciones.
- ▶ **role**: Define el rol del elemento en aplicaciones accesibles, ayudando a tecnologías de asistencia a interpretar su propósito.
- ▶ **aria-***: Atributos ARIA que mejoran la accesibilidad, proporcionando información adicional sobre el comportamiento y estado del elemento.



Ejemplo 6

Ejercicio 10

- ▶ Crea una página web en HTML que cumpla con los siguientes requisitos:
 - ▶ Formulario de Contacto:
 - ▶ Diseña un formulario de contacto que incluya los siguientes campos:
 - ▶ Nombre (input de texto)
 - ▶ Correo electrónico (input de email)
 - ▶ Mensaje (textarea)
 - ▶ Botón de envío
 - ▶ Uso de Atributos Globales:
 - ▶ Utiliza el atributo id para identificar cada campo y etiqueta del formulario.
 - ▶ Aplica clases (class) para estilizar los campos y el botón (puedes definir clases como campo-formulario y boton-enviar).
 - ▶ Añade un atributo title a cada campo para proporcionar información adicional cuando el usuario pase el cursor sobre ellos (por ejemplo, "Introduce tu nombre completo").
 - ▶ Usa lang y dir si incluyes texto en otro idioma o si necesitas cambiar la dirección del texto.
 - ▶ Implementa data-* para almacenar información personalizada en el botón de envío (por ejemplo, data-accion="enviar-formulario").
 - ▶ Utiliza accesskey y tabindex para mejorar la accesibilidad y navegación por teclado del formulario.
 - ▶ Añade el atributo hidden a un párrafo que muestre un mensaje de éxito tras el envío (este mensaje debe estar oculto inicialmente).

Ejercicio 11

- ▶ Desarrolla una página web en HTML que funcione como una galería de imágenes. La página debe cumplir con los siguientes requisitos:
 - ▶ Estructura de la Galería:
 - ▶ Crea una sección `<section>` que contenga varias imágenes representadas por elementos `<img>`.
 - ▶ Cada imagen debe estar dentro de un contenedor `<div>`.
 - ▶ Uso de Atributos Globales:
 - ▶ Asigna un atributo `id` único a cada contenedor `<div>` de las imágenes.
 - ▶ Aplica una clase común (`class`) a todos los contenedores para estilizar la galería.
 - ▶ Usa `title` en cada `<img>` para proporcionar una descripción de la imagen.
 - ▶ Añade atributos `data-*` a cada `<div>` para almacenar información como `data-imagen-id` y `data-descripcion`.
 - ▶ Implementa `tabindex` en los contenedores `<div>` para permitir la navegación por teclado entre las imágenes.
 - ▶ Utiliza `role` y `aria-label` para mejorar la accesibilidad de la galería.
 - ▶ Interactividad con JavaScript:
 - ▶ Al hacer clic en una imagen, muestra un cuadro de diálogo `<dialog>` que contiene la imagen en tamaño grande y su descripción.

CSS

CSS

- ▶ CSS es un lenguaje de hojas de estilo utilizado para describir la presentación de documentos escritos en HTML o XML.
- ▶ Separa el contenido de la estructura (HTML) de la presentación visual (CSS), lo que facilita el mantenimiento y la flexibilidad del diseño.
- ▶ Beneficios:
 - ▶ Mejora la apariencia y usabilidad de las páginas web.
 - ▶ Permite reutilizar estilos en múltiples páginas.
 - ▶ Facilita la adaptación del diseño para diferentes dispositivos y pantallas.

Integración CSS con HTML

- ▶ Estilos en línea (Inline Styles):
 - ▶ Se aplican directamente en el atributo style de un elemento HTML.
 - ▶ Uso: Recomendado solo para cambios rápidos o pruebas, no para estilos en producción.
- ▶ Hoja de estilo interna (Internal Stylesheet):
 - ▶ Se incluyen dentro de la etiqueta <style> en el <head> del documento HTML.
 - ▶ Uso: Útil para estilos específicos de una sola página.
- ▶ Hoja de estilo externa (External Stylesheet):
 - ▶ Se crea un archivo .css separado y se vincula con la etiqueta <link>.
 - ▶ Uso: Recomendado para proyectos grandes, permite reutilizar estilos y mejorar la gestión.

Sintaxis básica de CSS

- ▶ Estructura de una regla CSS:

```
selector {  
  propiedad: valor;  
}
```

- ▶ **Selector:** Indica a qué elementos HTML se aplicará el estilo.
- ▶ **Propiedad:** La característica del elemento que se quiere estilizar (color, tamaño, margen, etc.).
- ▶ **Valor:** El valor asignado a la propiedad.

Enlazar hojas de estilo externas

- ▶ **Uso de la etiqueta <link>:** Debe colocarse dentro del <head> del documento HTML.
- ▶ Atributos importantes:
 - ▶ **rel="stylesheet"**: Indica que es una hoja de estilo.
 - ▶ **href="styles.css"**: Ruta al archivo CSS.

```
<head>  
  <link rel="stylesheet" href="Ejemplo7.css">
```

Buenas prácticas

- ▶ **Separación de preocupaciones:** Mantén el HTML para el contenido y estructura, CSS para estilos y JavaScript para comportamiento.
- ▶ **Nombres significativos:** Utiliza nombres de clases e IDs que reflejen su función o propósito.
- ▶ **Organización del CSS:** Agrupa reglas relacionadas y comenta secciones para facilitar el mantenimiento.

Selectores básicos

- ▶ **Selector de elemento (etiqueta):** Aplica estilos a todas las etiquetas de un tipo específico.css

```
p {  
    line-height: 1.5;  
}
```

- ▶ **Selector de clase:** Aplica estilos a elementos con un atributo class específico.

```
<div class="contenedor">Contenido aquí</div>
```

```
.contenedor {  
    padding: 20px;  
}
```

Selectores básicos & comentarios

- ▶ **Selector de ID:** Aplica estilos a un elemento con un atributo id único.

```
<nav id="menu-principal">Menú</nav>
```

```
#menu-principal {  
  background-color: #f8f8f8;  
}
```

- ▶ **Comentarios en CSS:** Se escriben como `/* ...*/` y pueden ir tanto dentro como fuera de las reglas.

```
/* Este es un comentario */  
.clase {  
  color: red; /* Comentario inline */  
}
```

Otros selectores básicos

- ▶ **Selector de grupo:** El selector de grupo en CSS permite aplicar las mismas reglas de estilo a múltiples selectores separados por comas en una sola declaración.

```
h1, h2, h3 {  
  color: blue;  
}
```

- ▶ **Selector universal:** El selector universal (*) en CSS selecciona todos los elementos del documento para aplicarles estilos comunes.

```
* {  
  margin: 0;  
  padding: 0;  
}
```

Cascada y especificidad

- ▶ **Concepto de cascada:** Las reglas CSS se aplican en un orden específico; las últimas reglas pueden sobrescribir a las anteriores.
- ▶ **Especificidad:** Determina qué estilo se aplica cuando múltiples reglas coinciden con un elemento.
- ▶ Orden de especificidad (de menor a mayor):
 - ▶ Selectores de elemento.
 - ▶ Selectores de clase.
 - ▶ Selectores de ID.
 - ▶ Estilos en línea.

Ejercicio 12

- ▶ Crea una página HTML sencilla que contenga los siguientes elementos:
 - ▶ Un encabezado `<h1>` con el texto "Bienvenido a mi sitio web".
 - ▶ Tres párrafos `<p>` con algún texto de ejemplo.
 - ▶ Un enlace `<a>` con la clase `enlace` que lleva a "#".
- ▶ Aplica los siguientes estilos utilizando selectores CSS:
 - ▶ Utiliza un selector de tipo para establecer el color de texto rojo para todos los párrafos `<p>`.
 - ▶ Usa un selector de clase para subrayar el enlace con la clase `enlace`.
 - ▶ Aplica un selector universal para establecer el margen y el padding de todos los elementos en cero.

Ejercicio 13

- ▶ Diseña una página HTML que incluya:
 - ▶ Una lista ordenada `<ol>` con tres elementos `<li>`.
 - ▶ El primer `<li>` tiene un atributo `id="elemento1"`.
 - ▶ El segundo `<li>` tiene una clase especial.
- ▶ Aplica los siguientes estilos utilizando selectores CSS:
 - ▶ Usa un selector de ID para establecer el fondo de color amarillo al elemento con `id="elemento1"`.
 - ▶ Utiliza un selector de tipo para cambiar el color de texto a azul para los `<li>`.
 - ▶ Aplica un selector de clase para poner el texto en negrita para el elemento con la clase especial.

Ejemplos

```
1 <!-- HTML -->
2 <p id="parrafo" class="texto" style="color: blue;">Este es un párrafo.</p>
3
4 <!-- CSS -->
5 p {
6   color: red;
7 }
8
9 .texto {
10   color: green;
11 }
12
13 #parrafo {
14   color: orange;
15 }
```

```
1 <!-- HTML -->
2 <div id="caja" class="contenedor">
3   Contenido de la caja.
4 </div>
5
6 <!-- CSS -->
7 div {
8   background-color: yellow;
9 }
10
11 .contenedor {
12   background-color: green;
13 }
14
15 #caja {
16   background-color: red;
17 }
18
19 * {
20   background-color: blue;
21 }
```

```
<!-- HTML -->
<span class="resaltado" style="font-weight: normal;">Texto importante.</span>

<!-- CSS -->
span {
  font-weight: lighter;
}

.resaltado {
  font-weight: bold;
}
```

Ejemplos

```

1 <!-- HTML -->
2 <p id="parrafo" class="texto" style="color: blue;">Este es un párrafo.</p>
3
4 <!-- CSS -->
5 p {
6   color: red;
7 }
8
9 .texto {
10  color: green;
11 }
12
13 #parrafo {
14   color: orange;
15 }

```

Azul

```

1 <!-- HTML -->
2 <div id="caja" class="contenedor">
3   Contenido de la caja.
4 </div>
5
6 <!-- CSS -->
7 div {
8   background-color: yellow;
9 }
10
11 .contenedor {
12   background-color: green;
13 }
14
15 #caja {
16   background-color: red;
17 }
18
19 * {
20   background-color: blue;
21 }

```

Rojo

```

<!-- HTML -->
<span class="resaltado" style="font-weight: normal;">Texto importante.</span>

<!-- CSS -->
span {
  font-weight: lighter;
}

.resaltado {
  font-weight: bold;
}

```

Normal

Unidades de medida

- ▶ Las unidades de medida en CSS son fundamentales para definir el tamaño, espaciado y posición de los elementos en una página web. Comprender las diferentes unidades y cómo funcionan es esencial para crear diseños efectivos y responsivos.

Unidades de medida absolutas

- ▶ Las unidades absolutas se basan en medidas fijas y no dependen de otros elementos o del contexto de visualización. Son ideales cuando se necesita un tamaño exacto e invariable. Sin embargo, su uso puede limitar la adaptabilidad del diseño en diferentes dispositivos y tamaños de pantalla.
- ▶ Principales unidades absolutas:
 - ▶ **px (píxeles)**: Unidad más común en CSS que representa un punto en la pantalla. Un píxel es la unidad más pequeña de una imagen digital.
 - ▶ **cm (centímetros)**: Mide longitudes en centímetros.
 - ▶ **mm (milímetros)**: Mide longitudes en milímetros.
 - ▶ **in (pulgadas)**: Mide longitudes en pulgadas (1in = 2.54cm).
 - ▶ **pt (puntos)**: Unidad tipográfica tradicional, donde 1pt = 1/72 de pulgada.
 - ▶ **pc (picas)**: Unidad tipográfica, donde 1pc = 12pt.

Unidades de medida relativas

- ▶ Las unidades relativas se basan en otras medidas, como el tamaño de fuente del elemento padre o el tamaño de la ventana gráfica. Son esenciales para crear diseños flexibles y adaptativos.
- ▶ Principales unidades relativas:
 - ▶ **% (porcentaje)**: Representa un porcentaje relativo al valor del elemento padre.
 - ▶ Uso típico: Anchos y alturas de elementos, márgenes, paddings.
 - ▶ **em**: Relativo al tamaño de fuente del elemento padre. Por defecto, 1em equivale al tamaño de fuente heredado.
 - ▶ Uso típico: Tamaños de fuente, espaciados.
 - ▶ **rem (root em)**: Relativo al tamaño de fuente del elemento raíz (<html>), generalmente el tamaño de fuente predeterminado del navegador.
 - ▶ Uso típico: Tamaños de fuente, evitando problemas de acumulación con em.

Unidades de medida relativas

- ▶ Principales unidades relativas:
 - ▶ **vw (viewport width):** 1vw equivale al 1% del ancho de la ventana gráfica.
 - ▶ Uso típico: Diseño responsivo basado en el ancho de la pantalla.
 - ▶ **vh (viewport height):** 1vh equivale al 1% de la altura de la ventana gráfica.
 - ▶ Uso típico: Alturas de elementos en función de la altura de la pantalla.
 - ▶ **vmin y vmax:**
 - ▶ **vmin:** Equivale al valor menor entre vw y vh.
 - ▶ **vmax:** Equivale al valor mayor entre vw y vh.
 - ▶ Uso típico: Mantener proporciones en diferentes orientaciones.
 - ▶ **ex:** Relativo a la altura de la letra "x" en la fuente actual.
 - ▶ Uso típico: Raramente usado debido a inconsistencias entre fuentes.
 - ▶ **ch:** Relativo al ancho del carácter "0" (cero) en la fuente actual.
 - ▶ Uso típico: Definir anchos basados en caracteres monoespaciados.

Unidades absolutas vs Unidades relativas

▶ **Unidades Absolutas:**

▶ **Pros:**

- ▶ Proporcionan tamaños fijos y predecibles.
- ▶ Útiles para medios impresos.

▶ **Contras:**

- ▶ No son flexibles en diseños responsivos.
- ▶ Pueden causar problemas de accesibilidad.

▶ **Unidades Relativas:**

▶ **Pros:**

- ▶ Adaptativas a diferentes tamaños de pantalla y dispositivos.
- ▶ Mejoran la accesibilidad al permitir a los usuarios ajustar el tamaño de fuente.

▶ **Contras:**

- ▶ Pueden requerir cálculos más complejos.
- ▶ Pueden ser afectadas por el contexto y la herencia de estilos.

Cuando utilizar cada unidad

- ▶ **px:** Cuando se necesita precisión y consistencia en todos los navegadores. Sin embargo, para diseños modernos y responsivos, es recomendable limitar su uso.
- ▶ **em y rem:** Para tamaños de fuente y espaciados que deben escalar según el tamaño de fuente base. rem es preferible para evitar problemas de acumulación.
- ▶ **%:** Ideal para anchos y alturas que deben adaptarse al contenedor padre.
- ▶ **vw, vh, vmin, vmax:** Para elementos que deben ajustarse al tamaño de la ventana gráfica, como imágenes de fondo o secciones de pantalla completa.
- ▶ **Unidades físicas (cm, mm, in, pt, pc):** Principalmente para medios impresos. No se recomiendan para diseño web.

Ejemplo 7

Ejemplo de Unidades Absolutas y Relativas en CSS

Unidades Absolutas

width: 200px
height: 100px

width: 5cm
height: 2.5cm

width: 50mm
height: 25mm

width: 2in
height: 1in

width: 144pt
height: 72pt

width: 12pc
height: 6pc

Ejercicio 14

- ▶ Crea una página web que contenga una tarjeta de presentación personal. La tarjeta debe cumplir con los siguientes requisitos:
 - ▶ Utiliza unidades absolutas (px, cm, mm, in, pt, pc) para definir las dimensiones y estilos.
 - ▶ La tarjeta debe incluir:
 - ▶ Un contenedor `<div>` con:
 - ▶ Ancho (width) de 350px.
 - ▶ Altura (height) de 200px.
 - ▶ Borde (border) de 2mm sólido y color gris.
 - ▶ Relleno (padding) de 10pt.
 - ▶ Una imagen de perfil dentro del contenedor con:
 - ▶ Ancho y altura de 2in.
 - ▶ Borde redondeado al 50% para hacerlo circular.
 - ▶ Un encabezado `<h1>` con tu nombre, utilizando un tamaño de fuente (font-size) de 24pt.
 - ▶ Un párrafo `<p>` con una breve descripción, utilizando un tamaño de fuente de 15pt.

Ejercicio 15

- ▶ Desarrolla una página web que contenga una barra de navegación y una sección de contenido principal. La página debe cumplir con los siguientes requisitos:
 - ▶ Utiliza unidades relativas (% , em, rem, vw, vh, vmin, vmax) para definir las dimensiones y estilos.
- ▶ La barra de navegación (<nav>) debe:
 - ▶ Tener una altura de 10vh.
 - ▶ Ocupar el 100% del ancho de la ventana.
 - ▶ Contener enlaces de navegación con un tamaño de fuente (font-size) de 2vmin.
- ▶ La sección de contenido principal debe:
 - ▶ Tener un ancho del 80% del contenedor padre. Utilizar em o rem para establecer los tamaños de fuente de los encabezados y párrafos.
 - ▶ Añade contenido de ejemplo en la sección principal para visualizar los estilos aplicados.

Ejercicio 16

- ▶ Crea una página web que muestre una galería de imágenes dispuestas en cuadrícula. La página debe cumplir con los siguientes requisitos:
 - ▶ Utiliza una combinación de unidades absolutas y relativas para definir tamaños y márgenes.
 - ▶ La galería debe contener cuatro imágenes, cada una dentro de un contenedor `<div>`.
 - ▶ Especifica lo siguiente para cada imagen:
 - ▶ Ancho (width) de 30vmax.
 - ▶ Altura (height) de 200px.
 - ▶ Margen (margin) de 1rem alrededor de cada imagen.
 - ▶ Borde (border) de 0.5cm sólido y color negro.
 - ▶ Asegúrate de que las imágenes se ajusten correctamente al cambiar el tamaño de la ventana del navegador.
 - ▶ Usa CSS para añadir las imágenes (`background-image: url('imagen4.jpg');`)

Colores

- ▶ Los colores en CSS pueden definirse de varias formas:
 - ▶ **Nombre Predefinido:** Utilizando nombres de colores reconocidos por CSS, como red, yellow, blue, green, etc.
 - ▶ **Código Hexadecimal:** Utilizando el formato #RRGGBB o #RRGGBBAA para especificar colores mediante valores hexadecimales.
 - ▶ **RGB (Red, Green, Blue):** Especificando los valores de rojo, verde y azul en formato decimal.
 - ▶ **RGBA (Red, Green, Blue, Alpha):** Similar a RGB, pero incluye un valor de opacidad (alpha) que va de 0 (transparente) a 1 (opaco).
 - ▶ **HSL (Hue, Saturation, Lightness):** Define colores basándose en matiz, saturación y luminosidad.
 - ▶ **HSLA (Hue, Saturation, Lightness, Alpha):** Extiende HSL añadiendo un canal alfa para la opacidad.

Colores básicos

Color	Nombre	HEX	RGB
Negro	black	#000000	rgb(0, 0, 0)
Blanco	white	#ffffff	rgb(255,255,255)
Rojo	red	#ff0000	rgb(255, 0, 0)
Azul	blue	#0000ff	rgb(0, 0, 255)
Amarillo	yellow	#ffff00	rgb(255,255,0)
Gris	gray	#808080	rgb(128,128,128)
Verde	green	#008000	rgb(0,128,0)

Propiedades más utilizadas

- ▶ Las propiedades CSS relacionadas con el color y el fondo permiten personalizar la apariencia de los elementos HTML. Algunas de las más comunes son:
 - ▶ **color:** Define el color del texto.
 - ▶ Valores: RGB, HSL, HEX, nombre del color, RGBA, HSLA.
 - ▶ **background-color:** Establece el color de fondo de un elemento.
 - ▶ Valores: RGB, HSL, HEX, nombre del color, RGBA, HSLA, transparent.
 - ▶ **background-image:** Aplica una imagen de fondo a un elemento.
 - ▶ Valores: url("..."), none.
 - ▶ **background-repeat:** Controla la repetición de la imagen de fondo.
 - ▶ Valores: repeat, repeat-x, repeat-y, no-repeat.

Propiedades más utilizadas

- ▶ Las propiedades CSS relacionadas con el color y el fondo permiten personalizar la apariencia de los elementos HTML. Algunas de las más comunes son:
 - ▶ **background-attachment:** Determina el comportamiento de desplazamiento de la imagen de fondo.
 - ▶ Valores: scroll, fixed.
 - ▶ **background-position:** Especifica la posición inicial de la imagen de fondo.
 - ▶ Valores: Porcentajes, longitudes, left, center, right, top, bottom.
 - ▶ **background-size:** Controla el tamaño de la imagen de fondo.
 - ▶ Valores: auto, cover, contain, valores específicos.
 - ▶ **opacity:** Ajusta la opacidad del elemento (incluyendo su contenido e hijos).
 - ▶ Valores: Números entre 0 (transparente) y 1 (opaco).

Propiedades más utilizadas

- ▶ Las propiedades CSS relacionadas con el color y el fondo permiten personalizar la apariencia de los elementos HTML. Algunas de las más comunes son:
 - ▶ **background-repeat:** Controla la repetición de la imagen de fondo.
 - ▶ Valores:
 - ▶ **repeat:** Repite la imagen en ambas direcciones.
 - ▶ **repeat-x:** Repite la imagen solo horizontalmente.
 - ▶ **repeat-y:** Repite la imagen solo verticalmente.
 - ▶ **no-repeat:** No repite la imagen; se muestra una sola vez.

Propiedades más utilizadas

- ▶ Las propiedades CSS relacionadas con el color y el fondo permiten personalizar la apariencia de los elementos HTML. Algunas de las más comunes son:
 - ▶ **background-attachment:** Determina el comportamiento de desplazamiento de la imagen de fondo.
 - ▶ Valores:
 - ▶ **scroll:** La imagen de fondo se desplaza junto con el contenido.
 - ▶ **fixed:** La imagen de fondo permanece fija en su posición en la ventana gráfica.

Propiedades más utilizadas

- ▶ Las propiedades CSS relacionadas con el color y el fondo permiten personalizar la apariencia de los elementos HTML. Algunas de las más comunes son:
 - ▶ **background-position:** Especifica la posición inicial de la imagen de fondo.
 - ▶ Valores:
 - ▶ **Palabras clave:** left, center, right, top, bottom.
 - ▶ **Porcentajes:** Por ejemplo, 50% 50% posiciona la imagen en el centro.
 - ▶ **Longitudes:** Valores exactos utilizando unidades como píxeles (px).

Propiedades más utilizadas

- ▶ Las propiedades CSS relacionadas con el color y el fondo permiten personalizar la apariencia de los elementos HTML. Algunas de las más comunes son:
 - ▶ **background-size:** Controla el tamaño de la imagen de fondo.
 - ▶ Valores:
 - ▶ **auto:** Mantiene el tamaño original de la imagen.
 - ▶ **cover:** Escala la imagen para cubrir completamente el contenedor.
 - ▶ **contain:** Escala la imagen para que quepa completamente dentro del contenedor.
 - ▶ **Valores específicos:** Dimensiones exactas utilizando unidades de medida (píxeles, porcentajes, etc.).

Ejemplo 8

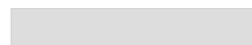
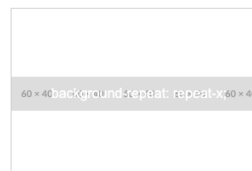
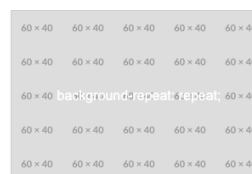


Ejemplo 9

http://127.0.0.1:5580/Clase3/Ejemplos/Ejemplo9.html



Ejemplos de Imágenes de Fondo en CSS



Ejercicio 17

- ▶ Crea una página web que contenga una tarjeta de presentación personal. La tarjeta debe cumplir con los siguientes requisitos:
 - ▶ Utiliza diferentes métodos para definir colores en CSS (nombre de color, hexadecimal, RGB, RGBA, HSL, HSLA).
 - ▶ La tarjeta debe incluir:
 - ▶ Un contenedor `<div>` con:
 - ▶ Un fondo de color definido en HSL.
 - ▶ Un borde de 2px sólido con color definido en RGB.
 - ▶ Un ancho (width) de 400px y un margen automático para centrarla horizontalmente.
 - ▶ Un encabezado `<h1>` con tu nombre, utilizando un color de texto definido por su nombre.
 - ▶ Un párrafo `<p>` con una breve descripción, utilizando un color de texto definido en hexadecimal.
 - ▶ Un botón `<button>` con el texto "Contacto", con:
 - ▶ Un fondo definido en RGBA para añadir transparencia.
 - ▶ Texto con color definido en HSLA.
 - ▶ Añade estilos para cambiar el color de fondo del botón al pasar el cursor por encima (hover), utilizando cualquier método de color.

Ejercicio 18

- ▶ Desarrolla una página de inicio para un sitio web que cumpla con los siguientes requisitos:
 - ▶ Aplica una imagen de fondo a toda la página utilizando la propiedad background-image y la URL <https://placeholder.co/1200x800>.
 - ▶ Configura las siguientes propiedades de fondo:
 - ▶ background-size: Utiliza el valor cover para que la imagen cubra toda el área visible.
 - ▶ background-attachment: Establece la imagen de fondo como fija (fixed) para lograr un efecto parallax al desplazarse.
 - ▶ background-position: Centra la imagen de fondo tanto horizontal como verticalmente.
 - ▶ Añade un título <h1> en el centro de la página con el texto "Bienvenido a Mi Sitio Web", utilizando:
 - ▶ Un color de texto definido en RGBA para añadir transparencia.
 - ▶ Una sombra de texto (text-shadow) utilizando colores definidos en hexadecimal.
 - ▶ Asegúrate de que el contenido esté centrado vertical y horizontalmente dentro de la ventana del navegador.

Ejercicio 19

- ▶ Crea una sección destacada en una página web que cumpla con los siguientes requisitos:
 - ▶ Utiliza un gradiente lineal `linear-gradient(dir, color1, color2)` como fondo de la sección, que vaya de un color a otro:
 - ▶ Color inicial definido en hexadecimal (`#ff7e5f`).
 - ▶ Color final definido en HSL (`hsl(240, 100%, 50%)`).
 - ▶ La sección debe tener:
 - ▶ Un ancho del 100% y una altura mínima (`min-height`) de 400px.
 - ▶ Un texto centrado tanto vertical como horizontalmente.
 - ▶ Dentro de la sección, incluye un encabezado `<h2>` y un párrafo `<p>` con el siguiente estilo:
 - ▶ El encabezado debe tener un color de texto definido en nombre de color.
 - ▶ El párrafo debe tener un color de texto definido en RGB.
 - ▶ Aplica un fondo semi-transparente al párrafo utilizando `background-color` con RGBA, para mejorar la legibilidad sobre el gradiente.

Ejercicio 20

- ▶ Crea un conjunto de tres botones estilizados que demuestren diferentes propiedades de fondo y color en CSS:
 - ▶ Botón 1: Efecto Hover con Cambio de Color de Fondo
 - ▶ Fondo inicial definido en RGB.
 - ▶ Al pasar el cursor por encima, el fondo cambia a un color definido en hexadecimal.
 - ▶ El texto del botón debe tener un color definido en HSL.
 - ▶ Botón 2: Fondo con Imagen y Ajuste de Tamaño
 - ▶ Aplica una imagen de fondo utilizando `background-image` con la URL `https://placeholder.co/100x50`.
 - ▶ Establece `background-size` a `contain` para ajustar la imagen dentro del botón.
 - ▶ El texto del botón debe ser legible sobre la imagen.
 - ▶ Botón 3: Degradado y Transparencia
 - ▶ Utiliza un degradado lineal como fondo que vaya de un color definido en RGBA a otro color definido en HSLA. `linear-gradient(dir, color1, color2)`
 - ▶ El degradado debe tener un ángulo de 45 grados.
 - ▶ El texto del botón debe tener un color contrastante y definido por su nombre de color.

Propiedades de texto

- ▶ Las propiedades de texto en CSS permiten controlar la apariencia y el formato del texto en una página web. A continuación, se presentan las propiedades de texto más utilizadas:
 - ▶ **text-indent:** Controla el desplazamiento de la primera línea de un bloque de texto, creando sangrías.
 - ▶ Valores: Longitud (como px, em) o porcentaje (%).
 - ▶ **text-align:** Determina la alineación horizontal del texto dentro de su contenedor.
 - ▶ Valores: left, right, center, justify.
 - ▶ **text-decoration:** Aplica efectos decorativos al texto, como subrayado, sobrelineado o tachado.
 - ▶ Valores: none, underline, overline, line-through.
 - ▶ **letter-spacing:** Ajusta el espacio entre caracteres individuales en el texto.
 - ▶ Valores: normal o una longitud específica.

Propiedades de texto

- ▶ Las propiedades de texto en CSS permiten controlar la apariencia y el formato del texto en una página web. A continuación, se presentan las propiedades de texto más utilizadas:
 - ▶ **word-spacing:** Controla el espacio entre palabras en el texto.
 - ▶ Valores: normal o una longitud específica.
 - ▶ **text-transform:** Modifica la capitalización del texto.
 - ▶ Valores: capitalize, uppercase, lowercase, none.
 - ▶ **line-height:** Establece la altura de las líneas de texto, afectando el interlineado.
 - ▶ Valores: Longitud, porcentaje o factor multiplicativo.
 - ▶ **vertical-align:** Ajusta la alineación vertical de elementos en línea o de tabla.
 - ▶ Valores: top, middle, bottom, baseline, sub, super, o una longitud específica.

Propiedades de texto

- ▶ Las propiedades de texto en CSS permiten controlar la apariencia y el formato del texto en una página web. A continuación, se presentan las propiedades de texto más utilizadas:
 - ▶ **vertical-align:** Ajusta la alineación vertical de elementos en línea o de tabla.
 - ▶ Valores:
 - ▶ **baseline:** Alinea el elemento con la línea base del texto circundante. Valor predeterminado para elementos en línea.
 - ▶ **top:** Alinea el borde superior del elemento con el de la línea más alta.
 - ▶ **middle:** Alinea el elemento verticalmente en el centro de la línea.
 - ▶ **bottom:** Alinea el borde inferior del elemento con el de la línea más baja.
 - ▶ **sub:** Desplaza el texto hacia abajo como subíndice.
 - ▶ **super:** Desplaza el texto hacia arriba como superíndice.

Propiedades de las fuentes

- ▶ Las propiedades CSS de las fuentes permiten controlar aspectos tipográficos como el tamaño, el tipo, el grosor y el estilo de las letras, entre otras características. Propiedades de las fuentes más destacadas:
 - ▶ **font-family:** Especifica la familia o tipo de fuente a utilizar en el texto.
 - ▶ Valores: Nombres de familias de fuentes específicas o genéricas.
 - ▶ **font-style:** Define el estilo de la fuente, como normal o cursiva.
 - ▶ Valores: normal, italic, oblique (similar a cursiva pero artificial).
 - ▶ **font-variant:** Controla la presentación de las letras, permitiendo mostrar el texto en versalitas.
 - ▶ Valores: normal, small-caps (minúsculas como mayúsculas de menor tamaño).
 - ▶ **font-weight:** Especifica el grosor o peso de los caracteres.
 - ▶ Valores: normal, bold, bolder, lighter, o valores numéricos de 100 a 900.
 - ▶ **font-size:** Controla el tamaño del texto.
 - ▶ Valores: Palabras clave, tamaños relativos, longitudes y porcentajes.

Propiedades de las fuentes

- ▶ Las propiedades CSS de las fuentes permiten controlar aspectos tipográficos como el tamaño, el tipo, el grosor y el estilo de las letras, entre otras características. Propiedades de las fuentes más destacadas:
 - ▶ **font-family:** Especifica la familia o tipo de fuente a utilizar en el texto.
 - ▶ Familias genéricas:
 - ▶ **serif:** Fuentes con remates en los caracteres (serifas), como "Times New Roman".
 - ▶ **sans-serif:** Fuentes sin remates, de aspecto más limpio, como "Arial" o "Helvetica".
 - ▶ **monospace:** Fuentes de ancho fijo, donde cada carácter ocupa el mismo espacio horizontal, como "Courier New".
 - ▶ **cursive:** Fuentes con estilo de escritura a mano, como "Comic Sans MS".
 - ▶ **fantasy:** Fuentes decorativas o estilizadas..

Propiedades de las fuentes

- ▶ **font-weight:** Especifica el grosor o peso de los caracteres.

- ▶ Valores:

- ▶ Palabras clave:

- ▶ **normal:** Peso normal de la fuente (equivalente a 400).
- ▶ **bold:** Negrita estándar (equivalente a 700).
- ▶ **bolder:** Un peso más grueso que el elemento padre.
- ▶ **lighter:** Un peso más ligero que el elemento padre.

- ▶ Valores numéricos:

- ▶ **100:** Ultra ligero.
- ▶ **200 a 300:** Ligero.
- ▶ **400:** Normal (estándar).
- ▶ **500:** Medio.
- ▶ **600:** Semi negrita.
- ▶ **700:** Negrita.
- ▶ **800 a 900:** Ultra negrita.

Propiedades de las fuentes

- ▶ **font-size:** Controla el tamaño del texto.
 - ▶ Palabras clave absolutas:
 - ▶ **xx-small**
 - ▶ **x-small**
 - ▶ **small**
 - ▶ **medium** (valor predeterminado)
 - ▶ **large**
 - ▶ **x-large**
 - ▶ **xx-large**
 - ▶ Palabras clave relativas:
 - ▶ **larger:** Un tamaño más grande que el elemento padre.
 - ▶ **smaller:** Un tamaño más pequeño que el elemento padre.
 - ▶ Longitudes absolutas y relativas.

Ejemplo 10

i http://127.0.0.1:5590/Class3/Ejemplos/Ejemplo10.html



Este párrafo tiene un **text-indent: 50px**, lo que desplaza la primera línea hacia la derecha.

Este texto está alineado a la izquierda usando **text-align: left**;

Este texto está centrado usando **text-align: center**;

Este texto está alineado a la derecha usando **text-align: right**;

Este texto está justificado usando **text-align: justify**; para que se alinee en ambos márgenes, ajustando el espacio entre palabras.

Este texto está subrayado usando **text-decoration: underline**;

Este texto tiene una línea sobre él usando **text-decoration: overline**;

Este texto está tachado usando **text-decoration: line-through**;

Este texto tiene un espaciado entre letras de 3px usando **letter-spacing: 3px**;

Este texto tiene un espaciado entre palabras de 10px usando **word-spacing: 10px**;

ESTE TEXTO ESTÁ EN MAYÚSCULAS USANDO **TEXT-TRANSFORM: UPPERCASE**;

este texto está en minúsculas usando **text-transform: lowercase**;

Este Texto Está Capitalizado Usando **Text-Transform: Capitalize**;

Este párrafo tiene un interlineado de 2 usando **line-height: 2**; lo que aumenta el espacio entre líneas para mejorar la legibilidad.

Este es un texto con un superíndice¹ y un subíndice₂ usando **vertical-align: super**; y **vertical-align: sub**;

Este texto utiliza una fuente serif: **font-family: 'Times New Roman', serif**;

Este texto utiliza una fuente sans-serif: **font-family: Arial, sans-serif**;

Este texto utiliza una fuente monospace: **font-family: 'Courier New', monospace**;

Este texto tiene estilo normal usando **font-style: normal**;

Este texto está en cursiva usando **font-style: italic**;

Este texto está en estilo oblicuo usando **font-style: oblique**;

Este texto es normal usando **font-variant: normal**;

ESTE TEXTO ESTÁ EN VERSALITAS USANDO **FONT-VARIANT: SMALL-CAPS**;

Este texto tiene peso normal usando **font-weight: normal**;

Este texto está en negrita usando **font-weight: bold**;

Este texto tiene un peso de 900 usando **font-weight: 900**;

Este texto es pequeño usando **font-size: 12px**;

Este texto es mediano usando **font-size: 16px**;

Este texto es grande usando **font-size: 24px**;

Ejercicio 21

- ▶ Diseña una página web que muestre un poema con estilos tipográficos específicos aplicando las propiedades de texto y fuentes en CSS. El poema debe cumplir con los siguientes requisitos:
 - ▶ Fuente y Tamaño:
 - ▶ Utiliza la fuente Georgia para el texto del poema.
 - ▶ Establece el tamaño de fuente en 20px.
 - ▶ Alineación y Sangría:
 - ▶ Centra el título del poema utilizando text-align.
 - ▶ Aplica una sangría de 40px a las primeras líneas de cada estrofa usando text-indent.
 - ▶ Estilo del Texto:
 - ▶ Convierte todo el texto del poema a cursiva utilizando font-style.
 - ▶ Resalta el título en negrita con font-weight.
 - ▶ Espaciado:
 - ▶ Ajusta el interlineado del poema a 1.8 usando line-height para mejorar la legibilidad.
 - ▶ Aumenta el espacio entre letras en el título a 2px con letter-spacing.
 - ▶ Decoración y Transformación:
 - ▶ Subraya el título del poema utilizando text-decoration.
 - ▶ Convierte el título a mayúsculas con text-transform.

Ejercicio 22

- ▶ Crea un menú de navegación horizontal para una página web y aplica estilos utilizando propiedades de texto y fuentes en CSS. El menú debe cumplir con los siguientes requisitos:
 - ▶ Elementos del Menú:
 - ▶ Inicio
 - ▶ Servicios
 - ▶ Productos
 - ▶ Contacto
 - ▶ Estilos Generales:
 - ▶ Utiliza una fuente sans-serif genérica para el menú.
 - ▶ Establece el tamaño de fuente en 16px.
 - ▶ Aplica un espaciado entre palabras de 15px con word-spacing para separar los elementos del menú.
 - ▶ Estilos Específicos:
 - ▶ Convierte los textos del menú a mayúsculas utilizando text-transform.
 - ▶ Aplica font-variant para mostrar los textos en versalitas.
 - ▶ Añade un efecto de subrayado solo al pasar el cursor por encima de cada elemento utilizando text-decoration y la pseudoclase :hover.
 - ▶ (Opcional) Interactividad:
 - ▶ Al hacer clic en un elemento del menú, el texto debe cambiar a negrita utilizando font-weight y mantener el estado activo.

Ejercicio 23

- ▶ Desarrolla la página de un artículo de blog aplicando propiedades de texto y fuentes en CSS para mejorar la presentación y legibilidad. El artículo debe cumplir con los siguientes requisitos:
 - ▶ Encabezados:
 - ▶ Los títulos (h1) deben estar alineados al centro y utilizar la fuente Arial.
 - ▶ Aplica un tamaño de fuente de 32px para los títulos y conviértelos a mayúsculas con text-transform.
 - ▶ Subtítulos:
 - ▶ Los subtítulos (h2) deben tener una fuente Verdana, tamaño de 24px y estar en cursiva.
 - ▶ Párrafos:
 - ▶ El texto del cuerpo debe utilizar la fuente Times New Roman con un tamaño de 18px.
 - ▶ Justifica los párrafos utilizando text-align.
 - ▶ Establece un interlineado de 1.5 con line-height.
 - ▶ Citas Destacadas:
 - ▶ Las citas deben estar dentro de un elemento <blockquote> y tener:
 - ▶ Fuente en cursiva.
 - ▶ Un borde izquierdo de 3px sólido y color gris.
 - ▶ Un margen izquierdo de 20px.
 - ▶ Enlaces:
 - ▶ Los enlaces dentro del artículo deben:
 - ▶ Tener un color azul y sin subrayado por defecto.
 - ▶ Al pasar el cursor, deben subrayarse utilizando text-decoration.

Recordatorio: Viewport

- ▶ En **web responsive** es necesario definir un viewport adecuado, de lo contrario es muy probable que la página no lea correctamente los media queries y que se vea en formato muy reducido, siendo necesario hacer zoom para ver el contenido.
- ▶ La etiqueta viewport se define en el elemento <head> del documento HTML y puede contener diferentes atributos para ajustar el comportamiento de la página en dispositivos móviles.

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

Recordatorio: Viewport

- ▶ En **web responsive** es necesario definir un viewport adecuado, de lo contrario es muy probable que la página no lea correctamente los media queries y que se vea en formato muy reducido, siendo necesario hacer zoom para ver el contenido.
- ▶ La etiqueta viewport se define en el elemento <head> del documento HTML y puede contener diferentes atributos para ajustar el comportamiento de la página en dispositivos móviles.

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

Recordatorio: Viewport

- ▶ Las propiedades más comunes utilizadas en content de la metaetiqueta viewport son:
 - ▶ **width:** Especifica el ancho inicial del viewport. Por ejemplo, `width=device-width` establece el ancho inicial del viewport para que coincida con el ancho del dispositivo.
 - ▶ **initial-scale:** Define el nivel de escala inicial del viewport. Por ejemplo, `initial-scale=1.0` establece que la página se mostrará inicialmente sin ninguna escala.
 - ▶ **minimum-scale y maximum-scale:** Estos atributos permiten establecer los límites mínimos y máximos de escala del viewport, controlando si el usuario puede hacer zoom en la página.
 - ▶ **user-scalable:** Este atributo permite habilitar o deshabilitar la capacidad del usuario para hacer zoom en la página. Si se establece en yes, el usuario podrá hacer zoom; si se establece en no, se deshabilitará el zoom.
 - ▶ **Height:** altura virtual de la pantalla o altura del viewport.

Pseudo-clases & pseudo-elementos

- ▶ Las **pseudo-clases** y los **pseudo-elementos** en CSS son herramientas poderosas que permiten aplicar estilos a elementos basados en su estado, posición o relación con otros elementos, incluso cuando no es posible seleccionarlos directamente con selectores normales.
- ▶ Las **pseudo-clases** se utilizan para definir un estado especial de un elemento. Por ejemplo, cuando el usuario pasa el cursor sobre un enlace, cuando un elemento es el primer hijo de su padre, o cuando un elemento está enfocado.
- ▶ Los **pseudo-elementos** permiten aplicar estilos a partes específicas de un elemento que no están disponibles a través de selectores normales, como la primera letra o línea, o insertar contenido antes o después de un elemento.

```
selector:pseudoclase {  
    /* Declaraciones de estilo */  
}
```

```
selector::pseudoelemento {  
    /* Declaraciones de estilo */  
}
```

Pseudo-clases

► Pseudoclasas Estructurales:

- **:first-child**: Selecciona el elemento que es el primer hijo de su padre.
- **:last-child**: Selecciona el elemento que es el último hijo de su padre.
- **:nth-child(n)**: Selecciona el elemento que es el enésimo hijo de su padre. n puede ser un número, una palabra clave o una expresión. Ejemplo: `:nth-child(2)` selecciona el segundo hijo.
- **:nth-last-child(n)**: Selecciona el enésimo hijo desde el final.
- **:only-child**: Selecciona el elemento si es el único hijo de su padre.
- **:first-of-type**: Selecciona el primer elemento de su tipo (etiqueta) dentro de su padre.
- **:last-of-type**: Selecciona el último elemento de su tipo dentro de su padre.
- **:nth-of-type(n)**: Selecciona el enésimo elemento de su tipo dentro de su padre.

Pseudo-clases

▶ Pseudoclasas de Enlace e Interacción:

- ▶ **:link:** Selecciona los enlaces que aún no han sido visitados.
- ▶ **:visited:** Selecciona los enlaces que ya han sido visitados.
- ▶ **:hover:** Selecciona el elemento cuando el usuario coloca el cursor sobre él.
- ▶ **:active:** Selecciona el elemento en el momento en que es activado (por ejemplo, al hacer clic).
- ▶ **:focus:** Selecciona el elemento que tiene el foco (por ejemplo, un campo de texto seleccionado).

▶ Pseudoclasas de Negación:

- ▶ **:not(selector):** Selecciona todos los elementos que no coinciden con el selector especificado. Ejemplo: `:not(.activo)` selecciona los elementos que no tienen la clase activo. Solo acepta un selector simple.

Pseudo-clases

▶ Pseudoclasas de Estado:

- ▶ **:enabled:** Selecciona los elementos que están habilitados (como inputs).
- ▶ **:disabled:** Selecciona los elementos que están deshabilitados.
- ▶ **:checked:** Selecciona los elementos que están marcados (como checkboxes o radios seleccionados).
- ▶ **:invalid:** Selecciona los elementos que no cumplen con la validación.
- ▶ **:valid:** Selecciona los elementos que cumplen con la validación.

▶ Pseudoclasas de Contenido:

- ▶ **:empty:** Selecciona los elementos que no tienen hijos (incluyendo texto).
- ▶ **:root:** Selecciona el elemento raíz del documento. En HTML, es el <html>.

▶ Pseudoclasas de Usuario:

- ▶ **:target:** Selecciona un elemento que coincide con el fragmento de la URL (lo que viene después de #).
- ▶ **:lang(language):** Selecciona los elementos que tienen establecido el atributo lang con el valor especificado.

Pseudo-elementos

- ▶ **::before:** Inserta contenido antes del contenido de un elemento.
- ▶ **::after:** Inserta contenido después del contenido de un elemento.
- ▶ **::first-letter:** Selecciona la primera letra de un elemento de bloque.
- ▶ **::first-line:** Selecciona la primera línea de texto de un elemento de bloque.
- ▶ **::selection:** Selecciona la porción del documento que ha sido seleccionada por el usuario.
- ▶ **::placeholder:** Selecciona el texto de marcador de posición de un input o textarea.
- ▶ **::backdrop:** Estiliza el fondo detrás de un elemento en pantalla completa.
- ▶ **::marker:** Estiliza el marcador de listas (puntos, números).

Variables en CSS

- ▶ Las variables CSS, son una característica introducida en CSS3 que permite a los desarrolladores almacenar y reutilizar valores en las hojas de estilo. Esto facilita la gestión y el mantenimiento del código.
- ▶ Se definen con un nombre personalizado y se pueden aplicar a cualquier propiedad CSS.
 - ▶ **Definición de una variable:** Se utiliza la sintaxis `--nombre-de-variable`.
 - ▶ **Uso de una variable:** Se emplea la función `var(--nombre-de-variable)`.
- ▶ Las variables se definen dentro de un selector, usualmente en el selector `:root` para que estén disponibles globalmente.

```
:root {  
  --color-primario: #3498db;  
  --espacio-base: 16px;  
}
```

```
h1 {  
  color: var(--color-primario);  
  margin-bottom: calc(var(--espacio-base) * 2);  
}
```

Variables en CSS

▶ Alcance de las Variables:

- ▶ **Variables Globales:** Definidas en :root o en un elemento de nivel superior, están disponibles en todo el documento.
- ▶ **Variables Locales:** Definidas dentro de un selector específico, solo están disponibles dentro de ese selector y sus elementos hijos.

▶ Valores por Defecto:

- ▶ La función var() permite especificar un valor por defecto en caso de que la variable no esté definida.

```
.elemento {  
  color: var(--color-secundario, #e74c3c);  
}
```

Variables en CSS

- ▶ Las variables CSS pueden ser manipuladas con JavaScript para cambiar estilos dinámicamente.

```
function cambiarTema() {  
  const root = document.documentElement;  
  root.style.setProperty('--color-primario', '#8e44ad');  
  root.style.setProperty('--color-secundario', '#f1c40f');  
}
```

Ejemplo 11

http://127.0.0.1:5500/Clase4/Ejemplos/Ejemplo11.html#seccion3



Ejemplos de Pseudoclasas y Pseudoelementos en CSS

Pseudoclasas de Enlace e Interacción

[Ir a Sección 1](#)

[Ir a Sección 2](#)

[Ir a Sección 3](#)

Pseudoclase :focus

Campo de texto enfocable:

Escribe aquí...

Pseudoclasas Estructurales

- Primer elemento (first-child)
- Segundo elemento (nth-child(2))
- Tercer elemento
- Penúltimo elemento (nth-last-child(2))
- Último elemento (last-child)
- Único elemento (only-child)

Primer párrafo (first-of-type)

Segundo párrafo (nth-of-type(2))

Último párrafo (last-of-type)

Pseudoclasas de Estado

Botón habilitado Botón deshabilitado

☐ Opción seleccionable

Correo electrónico (campo requerido):

correo@ejemplo.com

Pseudoclase :not()

Ejercicio 24

- ▶ Crea una tarjeta de perfil de usuario utilizando HTML y CSS que cumpla con los siguientes requisitos:
 - ▶ Variables CSS:
 - ▶ Colores: `--color-primario`, `--color-secundario`, `--color-fondo`, `--color-texto`.
 - ▶ Espaciado: `--espacio-base`.
 - ▶ Pseudoclases:
 - ▶ Al pasar el cursor sobre la tarjeta (`:hover`), cambia el color de fondo utilizando una variable CSS.
 - ▶ Cuando se haga clic en el botón "Seguir" (`:active`), cambia su color de fondo.
 - ▶ Estiliza los enlaces dentro de la tarjeta para que muestren un subrayado solo al pasar el cursor (`:hover`).
 - ▶ Pseudoelementos:
 - ▶ Utiliza `::before` para añadir un icono o texto antes del nombre de usuario.
 - ▶ Aplica `::after` para añadir una flecha o indicador después del botón "Seguir".
 - ▶ Usa `::first-letter` para resaltar la primera letra de la biografía del usuario.
 - ▶ Contenido de la tarjeta:
 - ▶ Imagen de perfil
 - ▶ Nombre de usuario: "Juan Pérez"
 - ▶ Biografía: "Desarrollador web apasionado por la tecnología y el código abierto."
 - ▶ Botón: "Seguir"

Ejercicio 25

- ▶ Desarrolla una lista de productos para una tienda en línea aplicando pseudoclases, pseudoelementos y variables CSS:

- ▶ **Variables CSS:**

- ▶ Define variables para colores y espaciados en :root.
- ▶ Crea variables para el precio original y el precio con descuento.

- ▶ **Pseudoclases:**

- ▶ Utiliza :nth-child(even) y :nth-child(odd) para alternar el fondo de los elementos de la lista.
- ▶ Aplica estilos diferentes a los productos que tengan la clase agotado utilizando :not().
- ▶ Al pasar el cursor sobre un producto (:hover), muestra una sombra alrededor del elemento.

- ▶ **Pseudoelementos:**

- ▶ Utiliza ::before para indicar si un producto es "Nuevo".
- ▶ Aplica ::after para mostrar un mensaje "Agotado" en productos que tengan la clase agotado.
- ▶ Usa ::first-line para estilizar la primera línea de la descripción del producto.

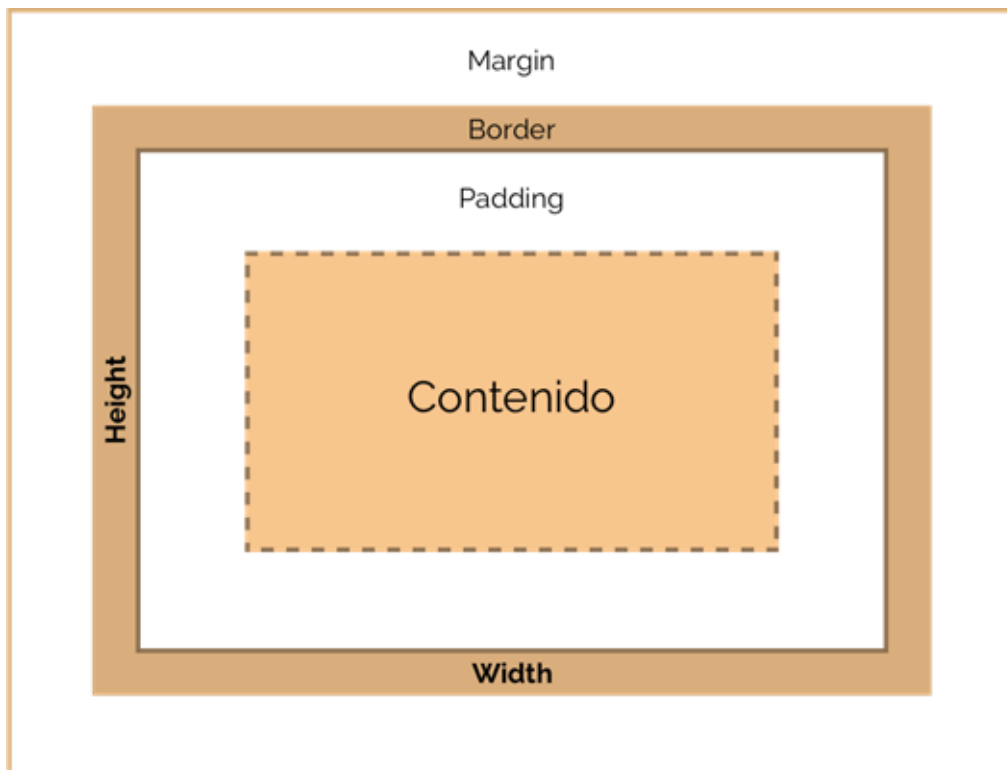
- ▶ **Estructura de la lista:**

- ▶ Producto 1: Disponible
- ▶ Producto 2: Agotado
- ▶ Producto 3: Nuevo
- ▶ Producto 4: Disponible

Ejercicio 26

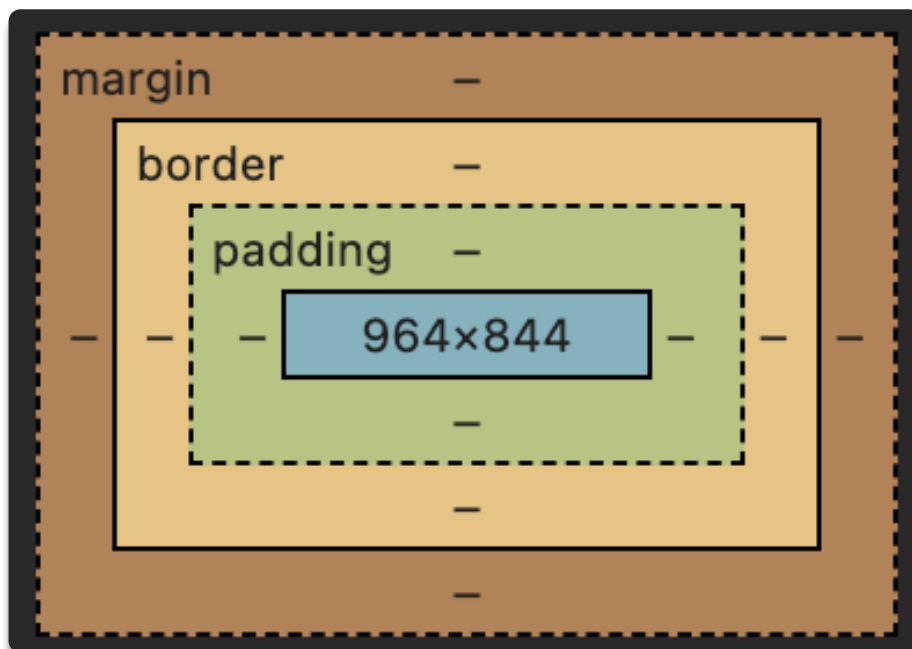
- ▶ Crea un formulario de contacto que incluya los siguientes elementos y estilos:
 - ▶ Variables CSS:
 - ▶ Define variables para colores, tamaños de fuente y espaciados.
 - ▶ Pseudoclases:
 - ▶ Estiliza los campos de entrada cuando están enfocados (:focus) cambiando el borde y el fondo.
 - ▶ Muestra un mensaje de validación en los campos requeridos que estén vacíos utilizando :invalid.
 - ▶ Cambia el estilo del botón de envío cuando el formulario es válido utilizando :valid.
 - ▶ Pseudoelementos:
 - ▶ Utiliza ::placeholder para estilizar el texto de marcador de posición en los campos de entrada.
 - ▶ Aplica ::after en las etiquetas de los campos requeridos para añadir un asterisco (*) indicando que son obligatorios.
 - ▶ Usa ::selection para cambiar el color de fondo y texto del contenido seleccionado por el usuario.
 - ▶ Elementos del formulario:
 - ▶ Nombre (campo de texto, requerido)
 - ▶ Correo electrónico (campo de email, requerido)
 - ▶ Mensaje (área de texto, requerido)
 - ▶ Botón de envío

Modelo de caja (Box Model)



- ▶ Todos los elementos en CSS se representan como cajas rectangulares.
- ▶ Propiedades clave:
 - ▶ **width y height:** Ancho y alto del contenido.
 - ▶ **padding:** Espacio entre el contenido y el borde.
 - ▶ **border:** Borde alrededor del padding y contenido.
 - ▶ **margin:** Espacio fuera del borde que separa el elemento de otros.

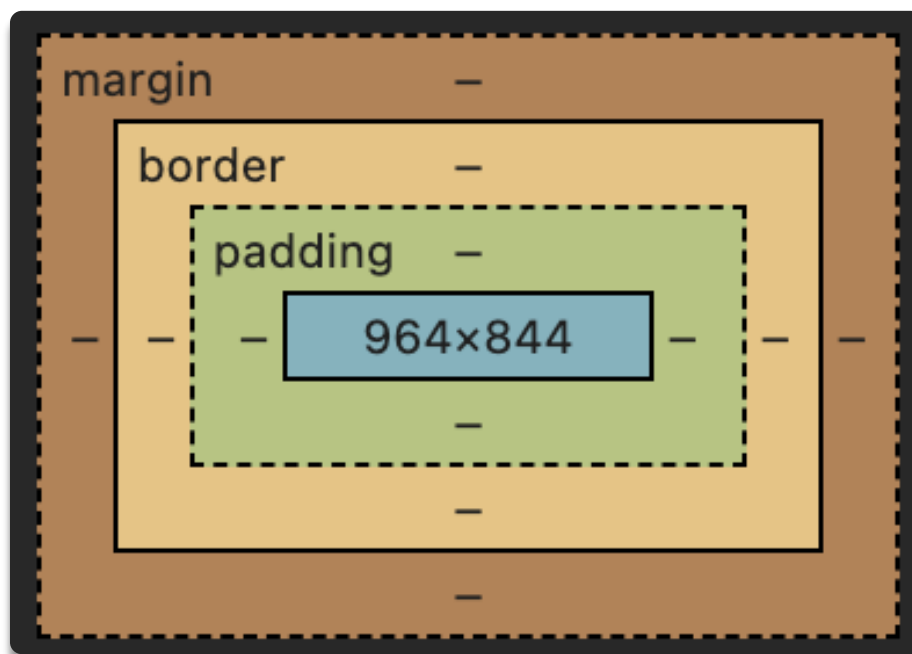
Modelo de caja (Box Model)



- ▶ **padding:** Espacio entre el contenido y el borde. Aumenta el tamaño visual de la caja sin afectar a su posición respecto a otros elementos.
 - ▶ **padding-top, padding-right, padding-bottom, padding-left:** Especifican el relleno en cada lado.
 - ▶ **padding:** Atajo para definir los cuatro valores en una sola propiedad.

```
/* Los valores se aplican en el orden: top, right, bottom, left */  
padding: 10px 15px 10px 15px;
```

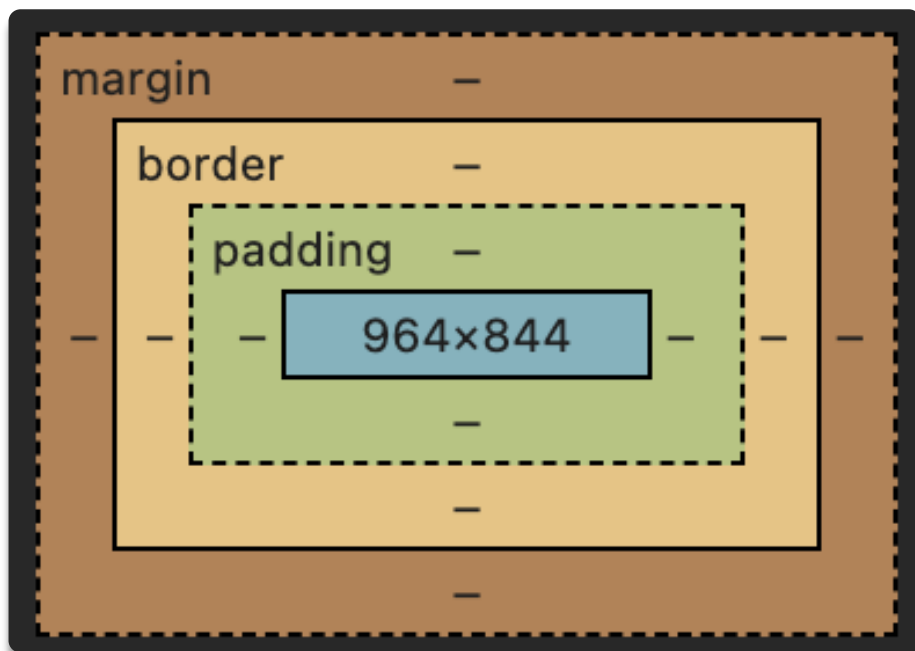
Modelo de caja (Box Model)



- ▶ **border**: Borde alrededor del padding y contenido.
 - ▶ **border-width**: Grosor del borde.
 - ▶ **border-style**: Estilo del borde (solid, dotted, dashed, etc.).
 - ▶ **border-color**: Color del borde.
 - ▶ **border**: Atajo para definir grosor, estilo y color.
 - ▶ **border-radius**: Curvatura del borde

```
border: 2px solid  #000;
```

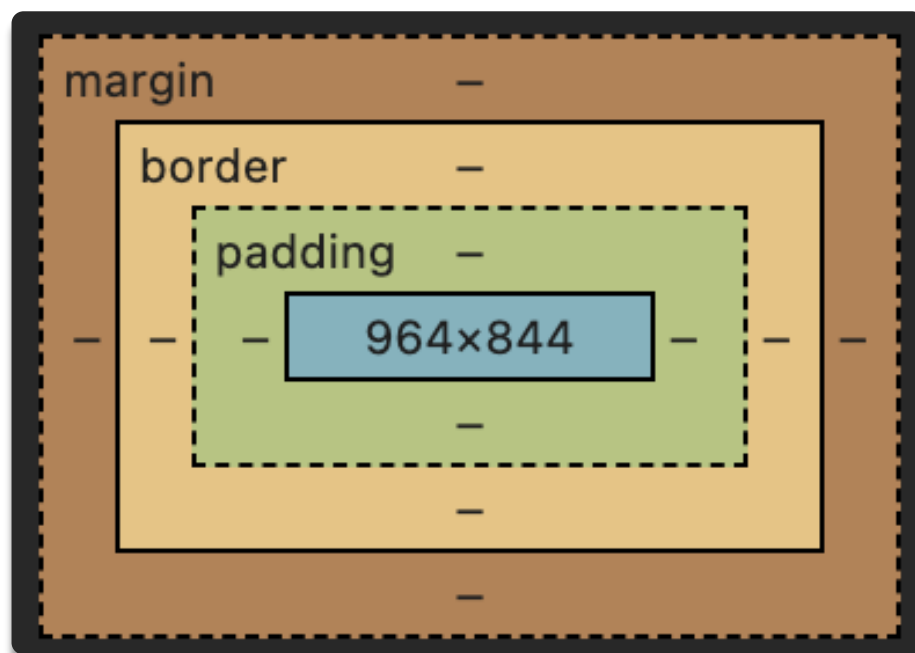
Modelo de caja (Box Model)



- ▶ **margin:** Espacio fuera del borde que separa el elemento de otros.
- ▶ **margin-top, margin-right, margin-bottom, margin-left:** Especifican el margen en cada lado.
- ▶ **margin:** Atajo para definir los cuatro valores en una sola propiedad.

```
/* Los valores se aplican en el orden: top, right, bottom, left */  
margin: 10px 15px 10px 15px;
```

Modelo de caja (Box Model)



- El modelo de java le dice al navegador si cuando indicamos el ancho, ese ancho incluye o no al padding y al borde. Por defecto no los incluye pero lo ideal es que el ancho si que incluya el padding y el borde, por eso hay que usar la propiedad "**box-sizing: border-box**" de CSS.

```
html {  
  box-sizing: border-box;  
}  
  
*, *:before, *:after {  
  box-sizing: inherit;  
}
```

Ejercicio 27

- ▶ Crea un botón utilizando HTML y CSS que cumpla con los siguientes requisitos:
 - ▶ Estructura del Botón:
 - ▶ Utiliza un elemento `<button>` o `<a>` con el texto "Enviar".
 - ▶ Estilos Aplicando el Modelo de Caja:
 - ▶ Ancho y Alto:
 - ▶ Establece un ancho fijo de 150px y una altura de 50px.
 - ▶ Relleno Interior (Padding):
 - ▶ Aplica un padding de 10px en todos los lados.
 - ▶ Margen Exterior (Margin):
 - ▶ Añade un margen superior e inferior de 20px y márgenes laterales automáticos para centrar el botón horizontalmente.
 - ▶ Borde (Border):
 - ▶ Añade un borde sólido de 2px de grosor y color azul #3498db.
 - ▶ Redondea las esquinas del borde con border-radius de 5px.
 - ▶ Color de Fondo y Texto:
 - ▶ Establece un color de fondo blanco #fff y un color de texto azul #3498db.

Ejercicio 28

- ▶ Crea un div que sea un cuadrado perfecto y esté centrado en la página, utilizando únicamente propiedades relacionadas con el modelo de caja.
 - ▶ Estructura del Cuadrado:
 - ▶ Utiliza un elemento `<div>` sin contenido interno.
 - ▶ Estilos Aplicando el Modelo de Caja:
 - ▶ Ancho y Alto:
 - ▶ Establece tanto el `width` como el `height` en 200px para crear un cuadrado.
 - ▶ Margen Exterior (Margin):
 - ▶ Aplica `margin: auto;` en los cuatro lados para centrar el cuadrado horizontal y verticalmente.
 - ▶ Nota: Dado que `margin: auto;` no centra verticalmente por defecto, utiliza `margin-top` y `margin-bottom` específicos si es necesario.
 - ▶ Borde (Border):
 - ▶ Añade un borde sólido de 3px de grosor y color negro `#000`.
 - ▶ Relleno Interior (Padding):
 - ▶ Aplica un `padding` de 20px en todos los lados.
 - ▶ Color de Fondo:
 - ▶ Establece un color de fondo gris claro `#e0e0e0`.

Arquitecturas CSS: BEM

- ▶ La arquitectura de CSS más utilizada es BEM (Block, Element, Modifier). BEM es una metodología que proporciona una convención para nombrar clases y estructurar el código CSS, lo que facilita la creación de interfaces de usuario escalables y mantenibles.
- ▶ La arquitectura CSS se refiere a las metodologías y prácticas utilizadas para organizar y estructurar el código CSS de manera que sea:
 - ▶ **Escalable:** Puede crecer sin volverse inmanejable.
 - ▶ **Mantenible:** Fácil de entender y modificar por diferentes desarrolladores.
 - ▶ **Reutilizable:** Componentes y estilos que se pueden reutilizar en diferentes partes del proyecto.

Arquitecturas CSS: BEM

- ▶ Además de BEM, existen otras arquitecturas y metodologías populares:
 - ▶ **OOCSS (Object-Oriented CSS)**: Se enfoca en separar la estructura de los elementos de su apariencia.
 - ▶ **SMACSS (Scalable and Modular Architecture for CSS)**: Propone una categorización de estilos en base a reglas, layout, módulos, estados y temas.
 - ▶ **ITCSS (Inverted Triangle CSS)**: Organiza el código CSS en capas, desde estilos genéricos hasta específicos.
 - ▶ **Atomic CSS**: Utiliza clases con una sola responsabilidad para aplicar estilos específicos.

Arquitecturas CSS: BEM

- ▶ Conceptos Básicos de BEM
 - ▶ **Bloque (Block):** Representa un componente independiente y autónomo.
 - ▶ Ejemplo: menu, button, header.
 - ▶ **Elemento (Element):** Parte de un bloque que tiene una función específica.
 - ▶ Sintaxis: bloque__elemento
 - ▶ Ejemplo: menu__item, button__icon.
 - ▶ **Modificador (Modifier):** Variación o estado del bloque o elemento.
 - ▶ Sintaxis: bloque--modificador o bloque__elemento--modificador
 - ▶ Ejemplo: button--primary, menu__item--active.

Arquitecturas CSS: BEM

```
<nav class="menu">
  <ul class="menu__list">
    <li class="menu__item menu__item--active">
      <a href="#" class="menu__link">Inicio</a>
    </li>
    <li class="menu__item">
      <a href="#" class="menu__link">Servicios</a>
    </li>
    <li class="menu__item">
      <a href="#" class="menu__link">Contacto</a>
    </li>
  </ul>
</nav>
```

```
.menu__list {
  list-style: none;
  margin: 0;
  padding: 0;
  display: flex;
}

.menu__item {
  margin-right: 20px;
}

.menu__link {
  color: #fff;
  text-decoration: none;
}

.menu__item--active .menu__link {
  font-weight: bold;
}
```

Ejemplo 12



Ejercicio 29

- ▶ Usando BEM con modificadores, haz el HTML y el CSS de los siguientes botones:



Ejercicio 30

- Reproduce utilizando BEM



Posicionamiento de Contenedores

- ▶ Los elementos de ordenación y las propiedades nos ayudan a organizar todos los componentes:
- ▶ **display:** Determina cómo se muestra un elemento en la página y cómo interactúa con otros elementos en el flujo del documento.
- ▶ **position:** Define cómo se posiciona un elemento en relación con su contenedor o el viewport, permitiendo colocarlo en ubicaciones específicas.
- ▶ **float:** Permite que un elemento se alinee a la izquierda o derecha dentro de su contenedor, haciendo que el contenido siguiente fluya alrededor de él.
- ▶ **clear:** Controla el flujo de los elementos después de un elemento flotante, especificando en qué lado no deben flotar.
- ▶ **z-index:** Establece el orden de apilamiento de elementos posicionados, determinando qué elementos se superponen sobre otros.
- ▶ **visibility:** Controla si un elemento es visible o invisible, manteniendo el espacio que ocuparía en el diseño de la página.

Posicionamiento de Contenedores: `display`

- ▶ **`display: block`**: El elemento se comporta como un bloque, ocupando todo el ancho disponible y comenzando en una nueva línea.
- ▶ **`display: inline`**: El elemento se comporta como un elemento en línea, ocupando solo el ancho necesario y permitiendo que otros elementos se coloquen a su lado en la misma línea.
- ▶ **`display: inline-block`**: Combina características de `block` e `inline`; el elemento no inicia una nueva línea y su tamaño puede ser ajustado.
- ▶ **`display: none`**: El elemento no se muestra en absoluto (no ocupa espacio en el documento).
- ▶ **`display: flex`**: Convierte el elemento en un contenedor flexible, permitiendo una distribución más dinámica de los elementos hijos.
- ▶ **`display: grid`**: Convierte el elemento en un contenedor de cuadrícula, facilitando el diseño de layouts complejos.

Posicionamiento de Contenedores: position

- ▶ **top, right, bottom, left:** Determinan la posición exacta del elemento.
- ▶ **position: static:** Valor por defecto; el elemento sigue el flujo normal del documento. Las propiedades **top, bottom, left y right** no están permitidas
- ▶ **position: relative:** El elemento es posicionado relativo a su posición normal.
- ▶ **position: absolute:** El elemento es posicionado relativo a su antecesor posicionado más cercano (que no sea static).
- ▶ **position: fixed:** El elemento es posicionado relativo a la ventana del navegador; permanece en la misma posición incluso al hacer scroll.
- ▶ **position: sticky:** El elemento se comporta como relative hasta que se alcanza una posición en el scroll, momento en el que se comporta como fixed.

Posicionamiento de Contenedores

- ▶ **float:** Permite que un elemento se alinee a la izquierda o derecha dentro de su contenedor, haciendo que el contenido siguiente fluya alrededor de él.
 - ▶ **float: left:** El elemento flota a la izquierda del contenedor.
 - ▶ **float: right:** El elemento flota a la derecha del contenedor.
 - ▶ **float: none:** Valor por defecto; el elemento no flota.
- ▶ **clear:** Controla el flujo de los elementos después de un elemento flotante, especificando en qué lado no deben flotar.
 - ▶ **left:** No permite elementos flotantes a la izquierda.
 - ▶ **right:** No permite elementos flotantes a la derecha.
 - ▶ **both:** No permite elementos flotantes a ninguno de los lados.
 - ▶ **none:** Valor por defecto; permite elementos flotantes a ambos lados.

Posicionamiento de Contenedores

- ▶ **z-index:** La propiedad z-index controla el orden en que se apilan los elementos que se superponen. Solo funciona en elementos posicionados (relative, absolute, fixed, sticky).
 - ▶ **Número entero:** Un número más alto coloca el elemento por encima de otros con valores más bajos.
 - ▶ **auto:** Valor por defecto.
- ▶ **visibility:** Controla si un elemento es visible o invisible, manteniendo el espacio que ocuparía en el diseño de la página.
 - ▶ **visible:** Valor por defecto; el elemento es visible.
 - ▶ **hidden:** El elemento es invisible, pero ocupa espacio.
 - ▶ **collapse:** En tablas, elimina una fila o columna; en otros elementos, se comporta como hidden.

Posicionamiento de Contenedores: Interacción

▶ **position y z-index**

- ▶ Solo los elementos posicionados pueden tener z-index.
- ▶ El z-index determina el orden de apilamiento de los elementos.

▶ **float y clear**

- ▶ Los elementos con float sacan al elemento del flujo normal, permitiendo que el texto y otros elementos fluyan a su alrededor.
- ▶ La propiedad clear se utiliza para evitar que los elementos siguientes fluyan alrededor de los elementos flotantes.

▶ **display y visibility**

- ▶ display: none elimina el elemento del documento, no ocupa espacio.
- ▶ visibility: hidden hace que el elemento sea invisible, pero mantiene su espacio.

Overflow

- ▶ Controla cómo se maneja el contenido que excede las dimensiones de un elemento, determinando si se oculta, se recorta o se muestran barras de desplazamiento para el contenido desbordado.
- ▶ Solo funciona sobre elementos de tipo bloque con una altura definida.
- ▶ Tiene variantes que permiten controlar el desbordamiento de contenido de forma independiente en cada eje:
 - ▶ **overflow-x:** Controla el desbordamiento horizontal del contenido.
 - ▶ **overflow-y:** Controla el desbordamiento vertical del contenido.
- ▶ Valores:
 - ▶ **visible:** El contenido desbordado es visible y no se recorta ni se oculta. Uso por defecto: Es el valor por defecto para overflow.
 - ▶ **hidden:** El contenido desbordado se recorta y no es visible.
 - ▶ **scroll:** Se añaden barras de desplazamiento, independientemente de si el contenido está desbordado o no.
 - ▶ **auto:** Se añaden barras de desplazamiento solo si el contenido está desbordado.

Ejemplo 13

i http://127.0.0.1:5500/Clase4/Ejemplos/Ejemplo13.html



Ejemplo de Posicionamiento y Ordenación

display: block, inline, inline-block, none

Elemento Block

Elemento Inline

Elemento Inline-Block

position: static, relative, absolute, fixed, sticky

Elemento Relativo

Etiqueta

Contenedor Absoluto

Elemento Absoluto

Elemento Sticky

Contenido desplazable...

Contenido desplazable...

float y clear

Float Left

Float Right

Elemento Fijo

Clear Both

Ejercicio 31

- Crea una página con un encabezado, tres secciones con texto, un menú lateral fijo a la derecha y un botón flotante en la esquina inferior derecha.



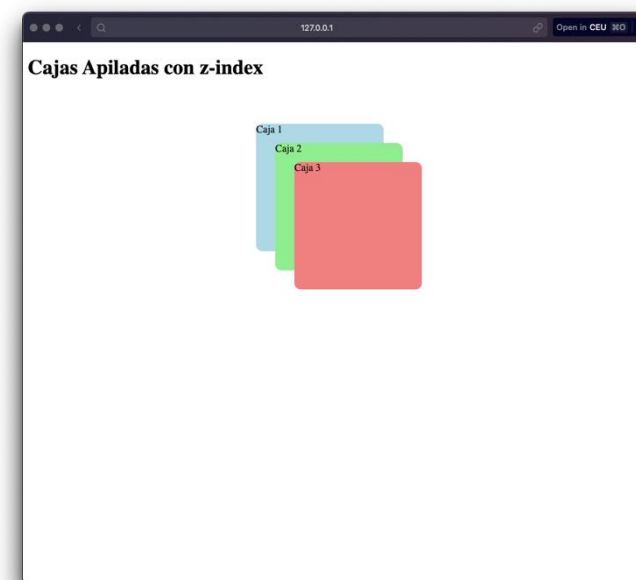
Ejercicio 32

- Crea una galería de imágenes con cajas flotantes que se organizan automáticamente en varias filas. Usa float para posicionar las imágenes y clear para organizar las filas.



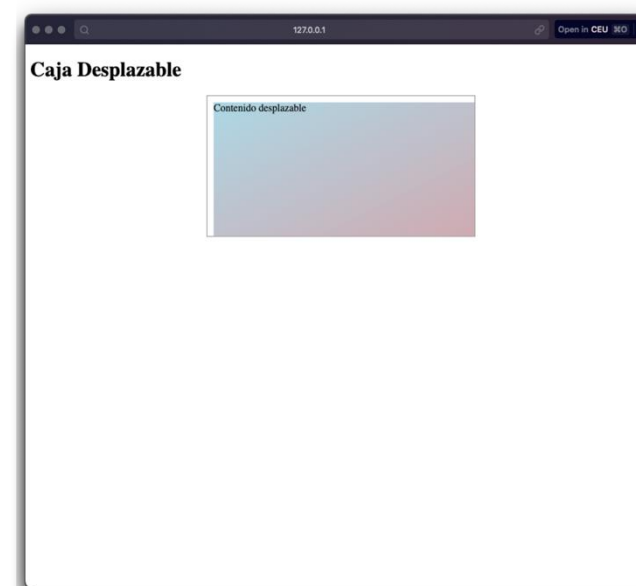
Ejercicio 33

- ▶ Crea tres cajas apiladas con diferentes colores y usa z-index para controlar cuál está encima.



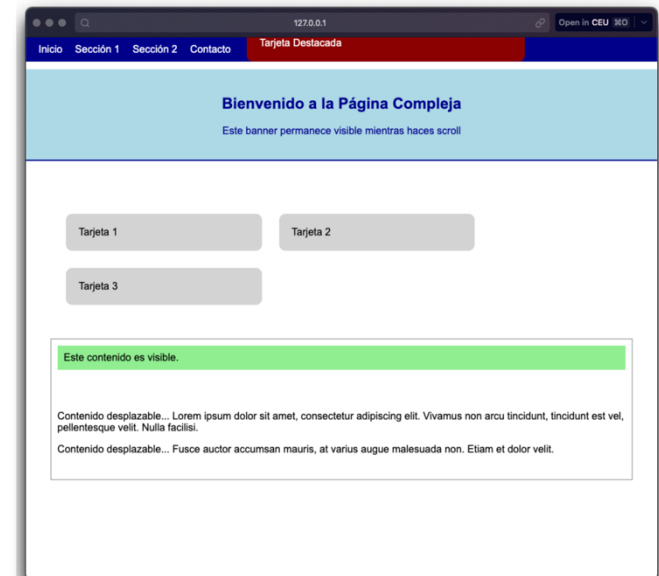
Ejercicio 34

- ▶ Crea un contenedor desplazable con barras de scroll horizontales y verticales.



Ejercicio 35

- ▶ Crea una página con las siguientes características:
 - ▶ Un menú fijo en la parte superior.
 - ▶ Un banner sticky que permanece visible al hacer scroll.
 - ▶ Una sección con tarjetas flotantes alineadas horizontalmente.
 - ▶ Una tarjeta con z-index más alto que se superpone sobre las otras.
 - ▶ Un contenedor con contenido desplazable y elementos visibles e invisibles.

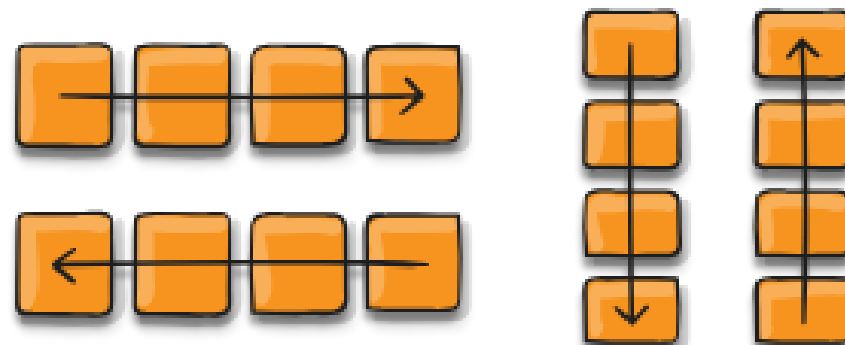


Flex

- ▶ **Flexbox** proporciona una manera eficiente y efectiva de diseñar, alinear y distribuir espacio entre elementos dentro de un contenedor, incluso cuando su tamaño es desconocido y/o dinámico. Flexbox facilita la creación de layouts flexibles y responsivos sin necesidad de utilizar floats o posicionamiento complejo.
- ▶ Elementos Clave:
 - ▶ **Contenedor Flex (flex container):** El elemento padre que tiene la propiedad `display: flex;` o `display: inline-flex;`.
 - ▶ **Ítems Flex (flex items):** Los elementos hijos directos del contenedor flex.
- ▶ Ejes en Flexbox:
 - ▶ **Eje Principal (main axis):** Dirección en la que se colocan los ítems (por defecto, horizontalmente de izquierda a derecha).
 - ▶ **Eje Transversal (cross axis):** Perpendicular al eje principal (por defecto, verticalmente de arriba hacia abajo).

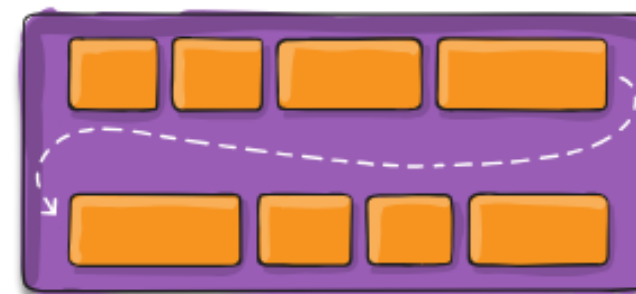
Flex: propiedades del contenedor

- ▶ **display:**
 - ▶ **flex:** Define un contenedor flex de bloque.
 - ▶ **inline-flex:** Define un contenedor flex en línea.
- ▶ **flex-direction:** Define la dirección del eje principal.
 - ▶ **row** (por defecto): De izquierda a derecha.
 - ▶ **row-reverse:** De derecha a izquierda.
 - ▶ **column:** De arriba hacia abajo.
 - ▶ **column-reverse:** De abajo hacia arriba.



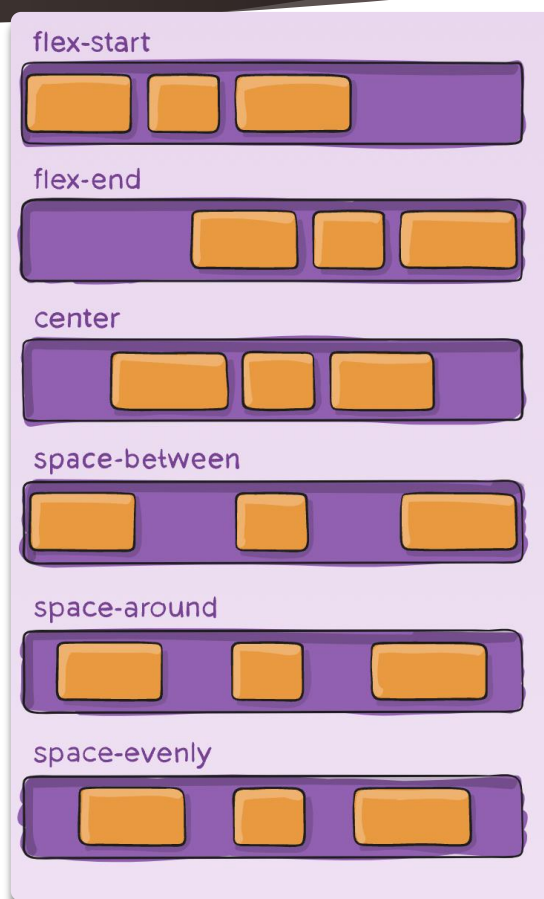
Flex: propiedades del contenedor

- ▶ **flex-wrap:** Controla si los ítems flex deben ajustarse o no al tamaño del contenedor.
 - ▶ **nowrap (por defecto):** Los ítems no se ajustan y se mantienen en una sola línea.
 - ▶ **wrap:** Los ítems se ajustan a la siguiente línea si no caben.
 - ▶ **wrap-reverse:** Igual que wrap, pero la nueva línea aparece encima de la anterior.
- ▶ **flex-flow:** Atajo para combinar flex-direction y flex-wrap.
 - ▶ `<flex-direction> || <flex-wrap>;`



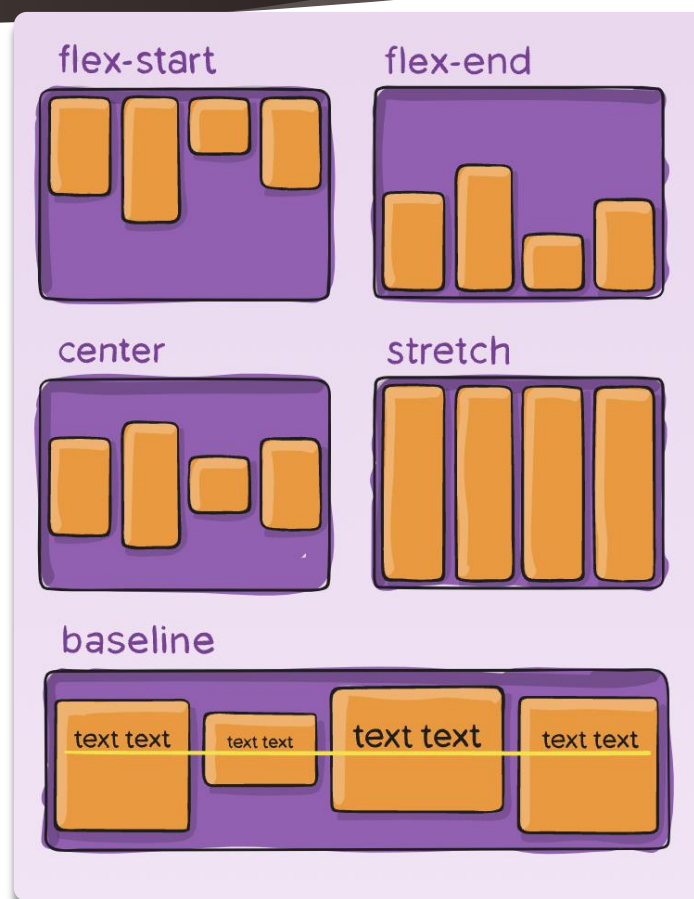
Flex: propiedades del contenedor

- ▶ **justify-content:** Alinea los ítems a lo largo del eje principal.
 - ▶ **flex-start (por defecto):** Alinea al inicio del eje.
 - ▶ **flex-end:** Alinea al final del eje.
 - ▶ **center:** Centra los ítems.
 - ▶ **space-between:** Distribuye el espacio sobrante entre los ítems.
 - ▶ **space-around:** Distribuye el espacio alrededor de los ítems.
 - ▶ **space-evenly:** Distribuye el espacio uniformemente entre los ítems.



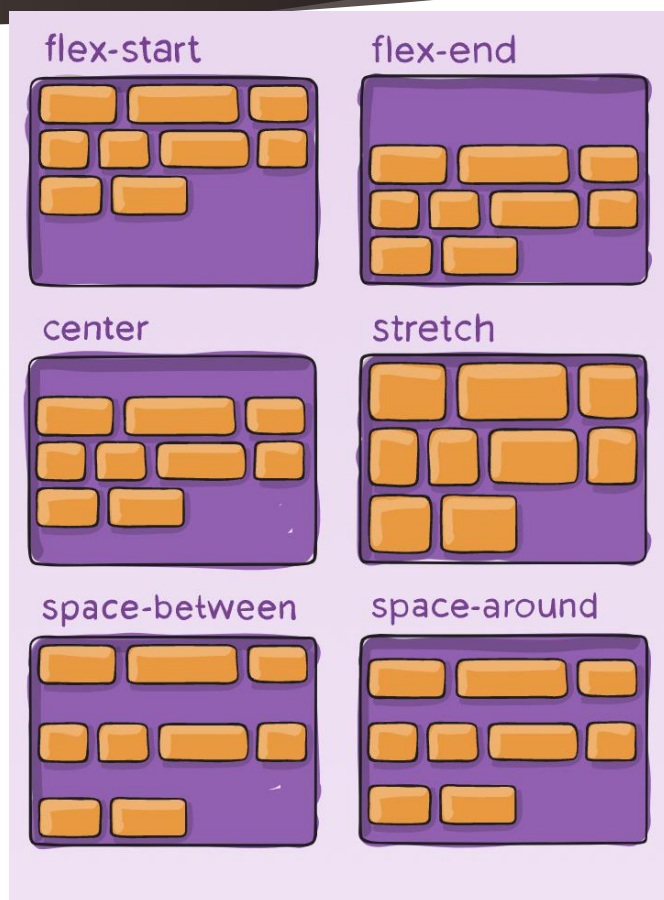
Flex: propiedades del contenedor

- ▶ **align-items:** Alinea los ítems a lo largo del eje transversal.
 - ▶ **stretch (por defecto):** Estira los ítems para llenar el contenedor.
 - ▶ **flex-start:** Alinea al inicio del eje transversal.
 - ▶ **flex-end:** Alinea al final del eje transversal.
 - ▶ **center:** Centra los ítems.
 - ▶ **baseline:** Alinea según la línea base del texto.



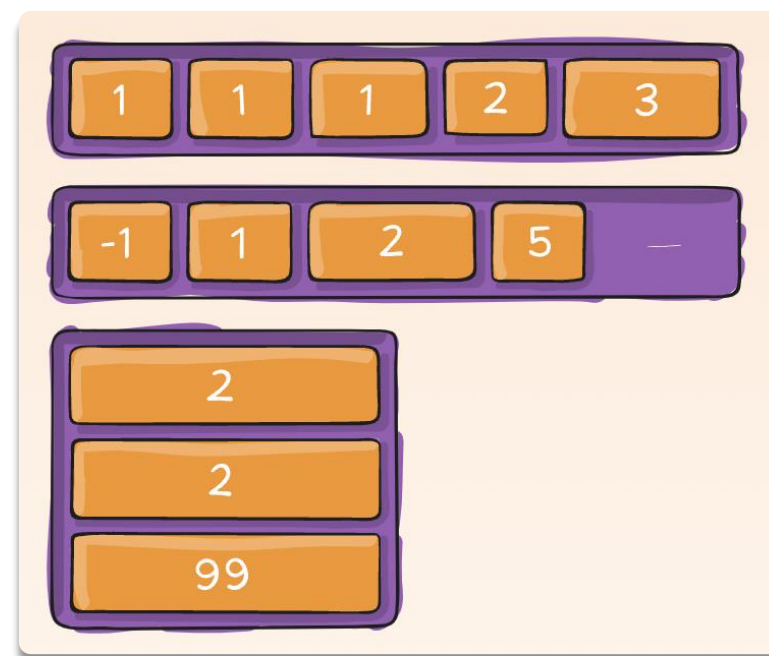
Flex: propiedades del contenedor

- ▶ **align-content:** Alinea las líneas flex cuando hay espacio extra en el eje transversal (aplica cuando hay múltiples líneas, es decir, con flex-wrap).
- ▶ **stretch (por defecto)**
- ▶ **flex-start**
- ▶ **flex-end**
- ▶ **center**
- ▶ **space-between**
- ▶ **space-around**
- ▶ **space-evenly**



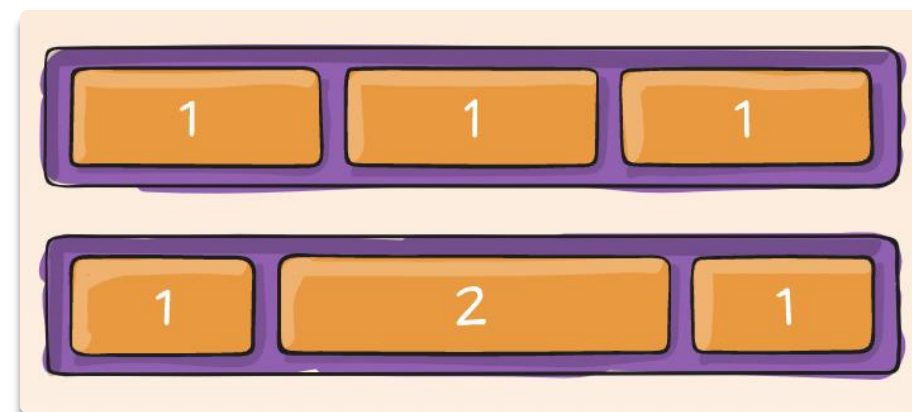
Flex: propiedades de los ítems

- ▶ **Order:** Define el orden en que se muestran los ítems.
 - ▶ Valor por defecto: 0
 - ▶ Valores: Enteros positivos o negativos.



Flex: propiedades de los items

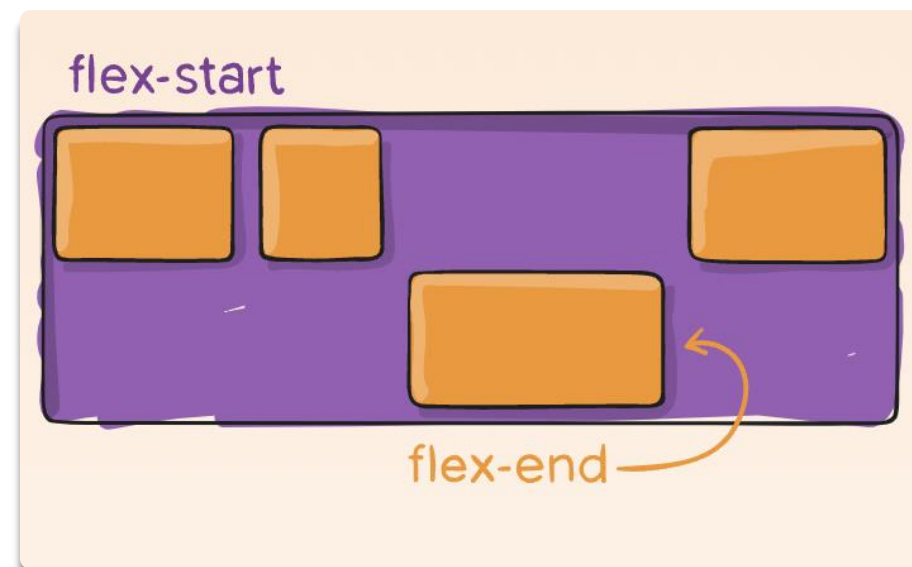
- ▶ **flex-grow:** Define la capacidad de un ítem para crecer si es necesario.
 - ▶ Valor por defecto: 0
 - ▶ Valores: Número positivo.
- ▶ **flex-shrink:** Define la capacidad de un ítem para encogerse si es necesario.
 - ▶ Valor por defecto: 1
 - ▶ Valores: Número positivo.
- ▶ **flex-basis:** Establece el tamaño inicial del ítem antes de distribuir el espacio.
 - ▶ Valores: Tamaños absolutos (px, em, %) o palabras clave (auto).
- ▶ **flex:** Atajo para combinar flex-grow, flex-shrink y flex-basis.
 - ▶ Sintaxis: flex: [flex-grow] [flex-shrink] [flex-basis];



flex-grow

Flex: propiedades de los items

- ▶ **align-self:** Permite sobrescribir el valor de align-items para un ítem específico.
 - ▶ **auto:** Coge el valor del padre
 - ▶ **stretch:** Estira el ítem para llenar el contenedor.
 - ▶ **flex-start:** Alinea al inicio del eje transversal.
 - ▶ **flex-end:** Alinea al final del eje transversal.
 - ▶ **center:** Centra los ítems.
 - ▶ **baseline:** Alinea según la línea base del texto.



Flex: Consejos y Buenas Prácticas

- ▶ **Evita Mezclar Flexbox con Floats:** Al diseñar con Flexbox, no es necesario utilizar float, ya que puede causar comportamientos inesperados.
- ▶ **Cuidado con el Eje Transversal:** Recuerda que propiedades como align-items afectan al eje transversal, que cambia según la dirección (flex-direction).
- ▶ **Usa atajos:** Propiedades como flex y flex-flow pueden simplificar tu código.
- ▶ **Conoce las Limitaciones:** Flexbox es un sistema unidimensional; para layouts bidimensionales complejos, considera usar CSS Grid.

Ejercicio 36

- ▶ Desarrolla una barra de navegación horizontal utilizando Flexbox que cumpla con los siguientes requisitos:
 - ▶ Estructura:
 - ▶ La barra de navegación debe contener un logotipo y un menú con enlaces.
 - ▶ El logotipo debe estar alineado a la izquierda.
 - ▶ El menú debe estar alineado a la derecha y sus elementos deben estar distribuidos horizontalmente.
 - ▶ Estilos:
 - ▶ La barra debe tener un fondo de color oscuro y los enlaces deben ser de color claro.
 - ▶ Al pasar el cursor sobre los enlaces, el color de fondo debe cambiar.

Ejercicio 37

- ▶ Crea un layout de tarjetas de productos utilizando Flexbox que cumpla con los siguientes requisitos:
 - ▶ Estructura:
 - ▶ Debe haber un contenedor que albergue varias tarjetas.
 - ▶ Cada tarjeta debe contener una imagen, un título y una descripción.
 - ▶ Estilos:
 - ▶ Las tarjetas deben organizarse en filas y ajustarse al ancho disponible.
 - ▶ Deben tener un espacio uniforme entre ellas.
 - ▶ Las tarjetas deben tener el mismo alto independientemente del contenido.

Ejercicio 38

- ▶ Utilizando Flexbox, centra un elemento tanto horizontal como verticalmente dentro de un contenedor que cumpla con los siguientes requisitos:
- ▶ Estructura:
 - ▶ Un contenedor que ocupa toda la ventana del navegador.
 - ▶ Un elemento interno, como un texto o imagen.
- ▶ Estilos:
 - ▶ El contenedor debe tener un color de fondo.
 - ▶ El elemento interno debe ser centrado sin utilizar márgenes o posicionamiento absoluto.

Ejercicio 39

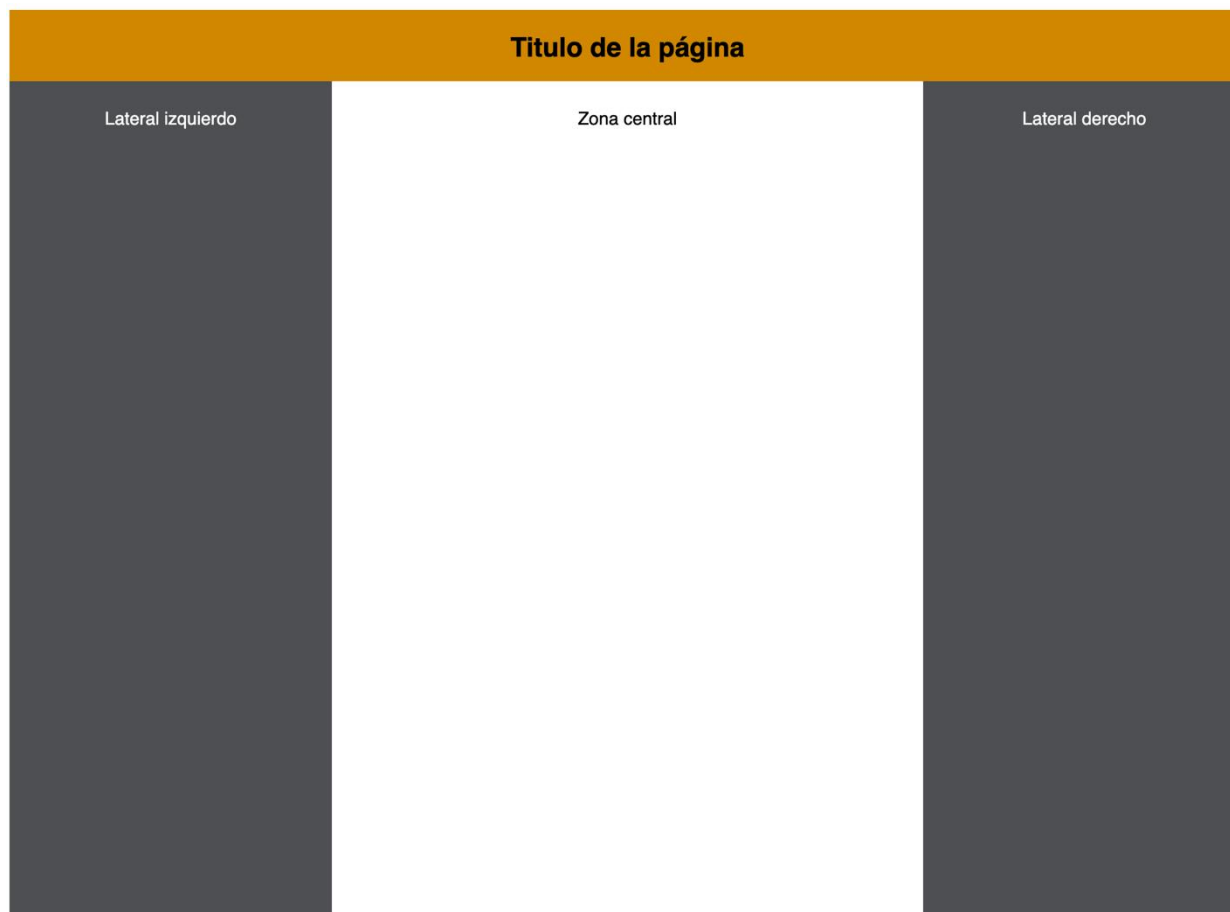
- ▶ Desarrolla una galería de imágenes utilizando Flexbox que cumpla con los siguientes requisitos:
- ▶ Estructura:
 - ▶ Un contenedor que alberga múltiples imágenes.
 - ▶ Las imágenes deben ajustarse al ancho disponible y mantener sus proporciones.
- ▶ Estilos:
 - ▶ Las imágenes deben distribuirse uniformemente en filas.
 - ▶ Debe haber un espacio uniforme entre las imágenes.
 - ▶ Al pasar el cursor sobre una imagen, ésta debe aumentar ligeramente de tamaño.

Ejercicio 40

- ▶ Crea una tabla de precios utilizando Flexbox que cumpla con los siguientes requisitos:
- ▶ Estructura:
 - ▶ Un contenedor que incluye tres planes de precios.
 - ▶ Cada plan debe contener un título, un precio, una lista de características y un botón.
- ▶ Estilos:
 - ▶ Los planes deben mostrarse en columnas de igual ancho y altura.
 - ▶ Debe haber un espacio uniforme entre las columnas.
 - ▶ El plan destacado debe tener estilos diferenciados.

Ejercicio 41

- ▶ Con el maquetado flexbox construye esta página donde todo el cuerpo es una estructura flex. Las columnas tienen una relación de anchuras 1:2:1. Coloca los textos centrados, y para dar este aspecto puedes poner una altura mínima igual a la altura del port view. El título está en negrita y tiene una altura de 24px, igual que el tamaño de la letra. Las columnas les puedes poner padding de 24px.



Ejercicio 42

- ▶ Se trata de utilizar los estilos flexbox para maquetar el cuerpo de la página en forma dos columnas, donde la columna derecha tiene a su vez dos filas iguales. Las columnas se reparten el ancho en la relación 2:1, es decir, columna derecha será la mitad de la izquierda. Para ver la estructura de la página como la imagen dar al body un padding de 20px, y a las columnas uno de 24px. El color del fondo en la imagen es #0e7c74, y las filas llevan bordes soldos de 1px en color lightgray.

Columna izquierda

Derecha parte alta

Derecha parte baja

Grid: Diseño de Cuadrícula en CSS

- ▶ **CSS Grid** es un sistema de diseño bidimensional que proporciona una forma más intuitiva y poderosa de crear estructuras de página complejas. A diferencia de Flexbox, que es un sistema unidimensional (trabaja en filas o columnas), CSS Grid trabaja en dos dimensiones, permitiendo controlar tanto filas como columnas simultáneamente.
- ▶ **Objetivo:** Facilitar la creación de layouts de cuadrícula, permitiendo un control preciso sobre la ubicación y el tamaño de los elementos.
- ▶ **Ventajas:**
 - ▶ Diseño de layouts complejos con menos código.
 - ▶ Control explícito sobre filas y columnas.
 - ▶ Mayor flexibilidad y alineación precisa de elementos.
 - ▶ Facilita la creación de diseños responsivos.

Grid: Conceptos básicos

▶ Elementos Clave

- ▶ **Contenedor Grid (grid container):** El elemento padre que tiene la propiedad `display: grid;` o `display: inline-grid;`.
- ▶ **Ítems Grid (grid items):** Los elementos hijos directos del contenedor grid.

▶ Terminología Importante

- ▶ **Líneas de la cuadrícula (grid lines):** Las líneas que dividen el grid en filas y columnas.
- ▶ **Pistas (tracks):** Filas y columnas del grid.
- ▶ **Celdas (cells):** El espacio entre dos líneas adyacentes, equivalente a una celda en una tabla.
- ▶ **Áreas de la cuadrícula (grid areas):** Espacio rectangular que abarca una o más celdas.

▶ Display:

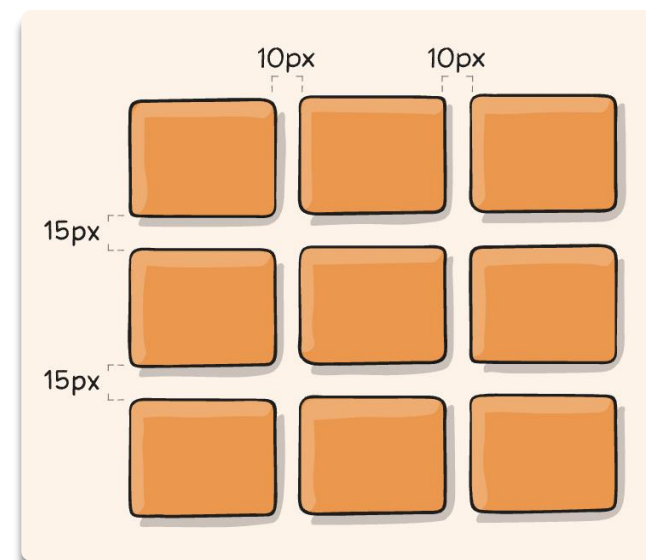
- ▶ **grid:** Define un contenedor grid de bloque.
- ▶ **inline-grid:** Define un contenedor grid en línea.

Grid: Propiedades contenedor

- ▶ **grid-template-columns y grid-template-rows:** Estas propiedades definen el número y tamaño de las columnas y filas del grid.
- ▶ Unidades de medida:
 - ▶ **px, %, em, rem:** Unidades absolutas y relativas.
 - ▶ **fr:** Fracciones del espacio disponible.
 - ▶ **auto:** Tamaño basado en el contenido.
- ▶ Funciones:
 - ▶ **repeat():** Repite un patrón.
 - ▶ **minmax():** Establece un rango mínimo y máximo.
 - ▶ **fit-content():** Ajusta el tamaño al contenido con un límite máximo.

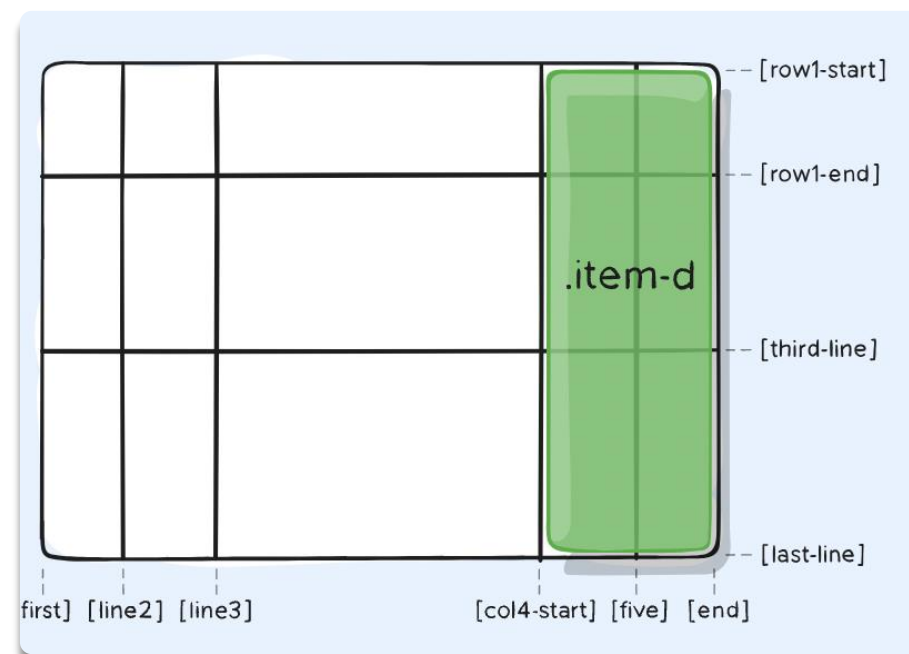
Grid: Propiedades contenedor

- ▶ **Espaciado entre Ítems**
 - ▶ **gap, column-gap, row-gap:** Estas propiedades controlan el espacio entre las filas y columnas.
 - ▶ **gap:** Espacio general entre filas y columnas.
 - ▶ **column-gap:** Espacio entre columnas.
 - ▶ **row-gap:** Espacio entre filas.



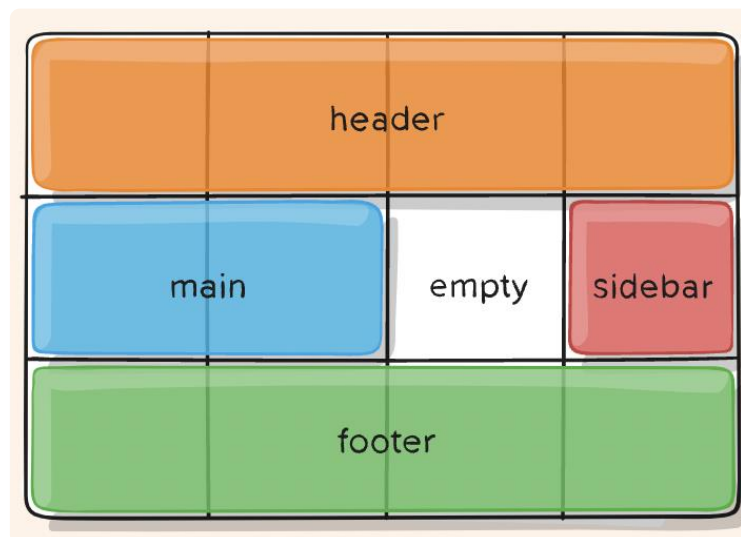
Grid: Propiedades contenedor

- ▶ **Posicionamiento elementos en el grid.**
 - ▶ **grid-column y grid-row:** Estas propiedades permiten posicionar un ítem en una o varias líneas de la cuadrícula.
 - ▶ Sintaxis: grid-column: inicio / fin;
 - ▶ Pueden usar valores numéricos o palabras clave como span para indicar cuántas columnas o filas debe abarcar.
 - ▶ **grid-area:** Combina grid-row y grid-column en una sola propiedad.



Grid: Propiedades contenedor

- ▶ Puedes asignar nombres a las líneas del grid para facilitar el posicionamiento.
- ▶ **grid-template-areas:** Permite crear un esquema visual del layout asignando nombres a áreas.



```
.contenedor {  
  display: grid;  
  grid-template-areas:  
    "header header header"  
    "sidebar main main"  
    "footer footer footer";  
  grid-template-columns: 1fr 2fr 1fr;  
  grid-template-rows: auto;  
}
```

```
.header {  
  grid-area: header;  
}
```

Grid: propiedades de los items

- ▶ Alineación en el Contenedor
 - ▶ **justify-items:** Alinea los ítems en el eje horizontal dentro de sus celdas.
 - ▶ **align-items:** Alinea los ítems en el eje vertical dentro de sus celdas.
 - ▶ **place-items:** Shorthand para combinar align-items y justify-items.
 - ▶ Valores comunes: start, end, center, stretch (por defecto).
- ▶ Alineación de Contenido
 - ▶ **justify-content:** Alinea el grid en el eje horizontal dentro del contenedor padre.
 - ▶ **align-content:** Alinea el grid en el eje vertical dentro del contenedor padre.
 - ▶ **place-content:** Shorthand para combinar align-content y justify-content.

Grid: propiedades de los items

► Alineación de Ítems Individuales

- **justify-self:** Alinea un ítem en el eje horizontal dentro de su celda.
- **align-self:** Alinea un ítem en el eje vertical dentro de su celda.
- **place-self:** Atajo para combinar align-self y justify-self.

.item-a		

.item-a		

Ejercicio 43

- ▶ Trata de construir esta tabla usando el display de tipo grid. Los elementos con el texto celda 1, celda 2... son bloques div que irán escritos en ese orden y con una clase que alinea el texto al centro. El gap o separación entre celdas es de 1px. Lo importante del ejercicio es aplicar la estructura con el layout grid y que tenga este aspecto.

celda 1	celda 2	celda 3
celda 4	celda 5	celda 6
celda 7	celda 8	celda 9

Ejercicio 44

- ▶ Trata de construir esta disposición tipo tabla usando el layout tipo grid. En el código HTML los elementos hijos son las celdas con el texto celda xx y están escritos como bloques div. La alineación del texto al centro es un estilo CSS con el color o el fondo. La línea entre celdas es un gap de 1px. Intenta lograrar el mismo aspecto que la figura.

celda 1		
celda 2	celda 3	celda 4
celda 5	celda 6	celda 7

Ejercicio 45

- ▶ Trata de construir esta disposición con celdas unidas entre si para formar ese sidebar con el texto celda2 y la cabecera con el texto celda 1. Usa el layout tipo grid. La primera columna tiene 100px de ancho y las restantes 150px cada una. En el código HTML las celdas con el texto celda xx son los elementos escritos como bloques div. La alineación del texto al centro es un estilo CSS con el color o el fondo. La línea entre celdas es un gap de 1px. Intenta lograrar el mismo aspecto que la figura.

celda 1		
celda 2	celda 3	celda 4
	celda 5	celda 6

Ejercicio 46

- Utiliza las áreas definidas en grid para construir esta disposición, similar al ejercicio anterior. En el código HTML las celdas con el texto celda xx son los elementos escritos como bloques div. El ancho de la primera columna es 100px y las restantes son 200px. La alineación del texto al centro es un estilo CSS con el color o el fondo. La línea entre celdas es un gap de 1px. Intenta lograrar el mismo aspecto que la figura.

celda 1		
celda 2	celda 3	celda 4
	celda 5	celda 6

Ejercicio 47

- ▶ Trata de construir esta disposición con celdas unidas entre si para formar ese sidebar derecho con el texto celda2 y la cabecera cn el texto celda 1. Usa el layout tipo grid. En el código HTML las celdas con el texto celda xx son los elementos escritos como bloques div. La alineación del texto al centro es un estilo CSS con el color o el fondo. La linea entre celdas es un gap de 1px. Intenta logarar el mismo aspecto que la figura.

celda 1			celda 2
celda 3	celda 4	celda 5	

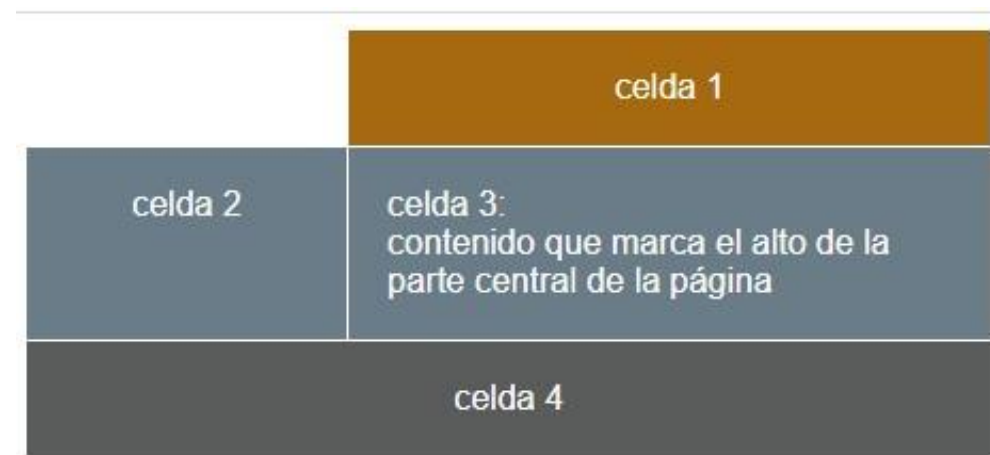
Ejercicio 48

- ▶ En esta estructura realizada con el layout grid se trata de combinar grupos de celdas. En el código HTML las celdas con el texto celda xx son los elementos escritos como bloques div. La alineación del texto al centro es un estilo CSS con el color o el fondo. La línea entre celdas es un gap de 1px. Intenta lograr el mismo aspecto que la figura.

celda 1	celda 2
celda 3	celda 4
celda 5	

Ejercicio 49

- ▶ La particularidad de este ejercicio es que el hueco que se ve en lo que sería la celda de la fila 1 y columna 1 está en blanco por que no existe. Realiza la distribución del contenido con el grid procurando mantener la relación de anchos. En el código HTML las celdas con el texto celda xx son los elementos escritos como bloques div. La alineación del texto al centro es un estilo CSS con el color o el fondo. La línea entre celdas es un gap de 1px. Intenta lograrar el mismo aspecto que la figura. En esta estructura realizada con el layout grid se trata de combinar grupos de celdas En el código HTML las celdas con el texto celda xx son los elementos escritos como bloques div. La alineación del texto al centro es un estilo CSS con el color o el fondo. La línea entre celdas es un gap de 1px. Intenta lograrar el mismo aspecto que la figura.



Ejercicio 50

- ▶ En el código HTML las celdas con el texto celda xx son elementos escritos como bloques div en orden: desde la celda 1 a la celda 6, pero en el layout deben aparecer en el orden que se muestra, es decir, la primera celda debe aparecer al final. Intenta lograrar el mismo aspecto que la figura, como ves todas las celdas tienen el mismo ancho .

celda 2	celda 3	celda 4
celda 5	celda 6	celda 1

Ejercicio 51

- ▶ En el código HTML las celdas con el texto celda xx son elementos escritos como bloques div en orden: desde la celda 1 a la celda 6, pero en el layout deben aparecer en el orden que se muestra, es decir, que la tabla se rellena por columnas. Como ves todas las celdas tienen el mismo ancho .

celda 1	celda 4
celda 2	celda 5
celda 3	celda 6

Ejercicio 52

- ▶ En esta distribución o cuadrícula grid la primera columna es mitad de ancho que la segunda, y existe otra tabla grid anidada en lo que sería la celda 4. Esta cuadrícula tiene una cabecera y dos columnas del mismo ancho.

celda 1	celda 2				
celda 3	Grid Anidado <table><tr><td>Lista 2</td><td>Lista 3</td></tr><tr><td>Lista 4</td><td>Lista 5</td></tr></table>	Lista 2	Lista 3	Lista 4	Lista 5
Lista 2	Lista 3				
Lista 4	Lista 5				
celda 5					

Sass

Sass

- ▶ Sass (Syntactically Awesome Stylesheets) es un preprocesador de CSS que extiende las capacidades del lenguaje CSS, añadiendo características avanzadas que facilitan la escritura y mantenimiento de hojas de estilo, especialmente en proyectos grandes y complejos.
- ▶ Sass permite escribir CSS de una manera más estructurada, eficiente y reutilizable, resolviendo algunas limitaciones del CSS puro. Después, el código Sass se compila en un archivo CSS estándar que los navegadores pueden interpretar.

Características principales de Sass

- ▶ **Variables:** Permite definir variables para valores reutilizables como colores, fuentes, tamaños, etc.
- ▶ **Nesting (Anidación):** Facilita la escritura de reglas anidadas, imitando la jerarquía del HTML.
- ▶ **Mixins:** Los mixins permiten reutilizar bloques de código y aceptar argumentos para mayor personalización.
- ▶ **Extends (Herencia):** Permite que un selector herede estilos de otro para evitar duplicar código.
- ▶ **Operaciones:** Sass permite realizar operaciones matemáticas directamente en los estilos.
- ▶ **Partials e Imports:** Divide tus estilos en archivos pequeños y organizados que se pueden importar en un archivo principal.

Características principales de Sass

- ▶ **Variables:** Permite definir variables para valores reutilizables como colores, fuentes, tamaños, etc.

SCSS Sass

```
$font-stack: Helvetica, sans-serif;
$primary-color: #333;

body {
  font: 100% $font-stack;
  color: $primary-color;
}
```

CSS

```
body {
  font: 100% Helvetica, sans-serif;
  color: #333;
}
```

Características principales de Sass

- **Nesting (Anidación):** Facilita la escritura de reglas anidadas, imitando la jerarquía del HTML.

SCSS Sass

```
nav {  
  ul {  
    margin: 0;  
    padding: 0;  
    list-style: none;  
  }  
  
  li { display: inline-block; }  
  
  a {  
    display: block;  
    padding: 6px 12px;  
    text-decoration: none;  
  }  
}
```

CSS

```
nav ul {  
  margin: 0;  
  padding: 0;  
  list-style: none;  
}  
nav li {  
  display: inline-block;  
}  
nav a {  
  display: block;  
  padding: 6px 12px;  
  text-decoration: none;  
}
```

Características principales de Sass

- **Mixins:** Los mixins permiten reutilizar bloques de código y aceptar argumentos para mayor personalización.

SCSS Sass

```
@mixin theme($theme: DarkGray) {  
  background: $theme;  
  box-shadow: 0 0 1px rgba($theme, .25);  
  color: #fff;  
}  
  
.info {  
  @include theme;  
}  
  
.alert {  
  @include theme($theme: DarkRed);  
}  
  
.success {  
  @include theme($theme: DarkGreen);  
}
```

CSS

```
.info {  
  background: DarkGray;  
  box-shadow: 0 0 1px rgba(169, 169, 169, 0.25);  
  color: #fff;  
}  
  
.alert {  
  background: DarkRed;  
  box-shadow: 0 0 1px rgba(139, 0, 0, 0.25);  
  color: #fff;  
}  
  
.success {  
  background: DarkGreen;  
  box-shadow: 0 0 1px rgba(0, 100, 0, 0.25);  
  color: #fff;  
}
```

Características principales de Sass

- **Extends (Herencia):** Permite que un selector herede estilos de otro para evitar duplicar código.

```
/* This CSS will print because %message-shared is extended. */
%message-shared {
  border: 1px solid #ccc;
  padding: 10px;
  color: #333;
}

// This CSS won't print because %equal-heights is never extended.
%equal-heights {
  display: flex;
  flex-wrap: wrap;
}
```

```
.message {
  @extend %message-shared;
}

.success {
  @extend %message-shared;
  border-color: green;
}

.error {
  @extend %message-shared;
  border-color: red;
}

.warning {
  @extend %message-shared;
  border-color: yellow;
}
```

Características principales de Sass

- **Operaciones:** Sass permite realizar operaciones matemáticas directamente en los estilos.

SCSS Sass

```
@use "sass:math";

.container {
  display: flex;
}

article[role="main"] {
  width: math.div(600px, 960px) * 100%;
}

aside[role="complementary"] {
  width: math.div(300px, 960px) * 100%;
  margin-left: auto;
}
```

CSS

```
.container {
  display: flex;
}

article[role=main] {
  width: 62.5%;
}

aside[role=complementary] {
  width: 31.25%;
  margin-left: auto;
}
```

Características principales de Sass

- **Partials e Imports:** Divide tus estilos en archivos pequeños y organizados que se pueden importar en un archivo principal.

SCSS Sass

```
// _base.scss
$font-stack: Helvetica, sans-serif;
$primary-color: #333;

body {
  font: 100% $font-stack;
  color: $primary-color;
}
```

```
// styles.scss
@use 'base';

.inverse {
  background-color: base.$primary-color;
  color: white;
}
```

CSS

```
body {
  font: 100% Helvetica, sans-serif;
  color: #333;
}

.inverse {
  background-color: #333;
  color: white;
}
```

Sintaxis

- ▶ Sass tiene dos sintaxis:
 - ▶ **Sass (Indentation-based syntax)**: Más conciso, no usa llaves {} ni punto y coma ;.
 - ▶ **SCSS (Sassy CSS)**: Similar a CSS, con llaves y punto y coma. Es la más usada actualmente.



CEU FP

Formación Profesional

Madrid

Bootstrap

Introducción al diseño responsive con Bootstrap



Utilizar Bootstrap

1. Incluye archivos css y js. Usa Popper.js que se puede incluir en la descarga o descargar aparte.
2. Versiones
 - a. Compilado.
 - i. <https://github.com/twbs/bootstrap/releases/download/v5.1.3/bootstrap-5.1.3-dist.zip>
 - b. Fuentes.
 - i. <https://github.com/twbs/bootstrap/archive/v5.1.3.zip>
3. Fuente
 - a. Descárgandolo de getbootstrap.com y copiándolo en nuestro proyecto
 - b. Cargándolo desde un CDN.
 - i. `<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.1.3/dist/css/bootstrap.min.css" rel="stylesheet" integrity="sha384-1BmE4kWBq78iYhFIdvKuhfTAU6auU8tT94WrHftjDbrCEXSU1oBoqyl2QvZ6jIW3" crossorigin="anonymous">`
 - ii. `<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.1.3/dist/js/bootstrap.bundle.min.js" integrity="sha384-ka7Sk0Gln4gmtz2MlQnikT1wXgYsOg+OMhuP+IIRH9sENBO0LRn5q+8nbTov4+1p" crossorigin="anonymous"></script>`

Dispositivos y navegadores

Móviles

	Chrome	Firefox	Safari	Android Browser & WebView
Android	Supported	Supported	—	v6.0+
iOS	Supported	Supported	Supported	—

Escritorio

	Chrome	Firefox	Microsoft Edge	Opera	Safari
Mac	Supported	Supported	Supported	Supported	Supported
Windows	Supported	Supported	Supported	Supported	—

Para tener soporte para Internet Explorer hay que usar la versión 4

Layout: Breakpoints

Los puntos de interrupción se basan en los tamaños de pantalla de los dispositivos.

Breakpoint	Class infix	Dimensions
X-Small	<i>None</i>	<576px
Small	<i>sm</i>	≥576px
Medium	<i>md</i>	≥768px
Large	<i>lg</i>	≥992px
Extra large	<i>xl</i>	≥1200px
Extra extra large	<i>xxl</i>	≥1400px

Layout: Containers (bootstrap01)

Los contenedores son el bloque fundamental que encierra y alinea nuestro contenido dentro de un dispositivo o un viewport. 3 tipos:

- .container, fija el ancho máximo para cada breakpoint
- .container-fluid, tiene el 100% de ancho en todos los breakpoints
- .container-{breakpoint}, que tiene el 100% en todos los puntos menos en el especificado

```
<div class="container-lg bg-primary">.container</div>
```

Los contenedores "responsive" (los que indican un breakpoint) cuando llegan a un tamaño de ventana más pequeño del que marcan, ocupan el 100%.

Los contenedores "fluid" siempre ocupan el 100%. `<div class="container-fluid">`

Práctica (bootstrap01.html)

En el siguiente ejemplo hemos creado diferentes containers con fondo de color. Crea una página, muéstrala en el navegador y cambia el tamaño de la ventana para ver el comportamiento.

```
<div class="container bg-primary">.container</div>
<div class="container-sm bg-secondary">.container-sm</div>
<div class="container-md bg-warning">.container-md</div>
<div class="container-lg bg-success">.container-lg</div>
<div class="container-xl bg-danger">.container-xl</div>
<div class="container-xxl bg-primary">.container-xxl</div>
<div class="container-fluid bg-secondary">.container-fluid</div>
```

Layout: Grid

Usa la rejilla flexbox "mobile-first" para construir diseños de todas las formas y tamaños gracias al sistema de 12 columnas y muchas clases predefinidas.

La rejilla de Bootstrap usa una serie de container, filas y columnas para formatear y alinear el contenido. Está construido sobre un flexbox y es completamente adaptable.

Para saber más sobre Flex:

<https://css-tricks.com/snippets/css/a-guide-to-flexbox/>

Grid: cómo funciona

- La rejilla soporta 6 breakpoints (xs, sm, md, lg, xl, xxl). De esta manera puedes controlar el comportamiento de los contenedores y las columnas para cada tamaño de pantalla. Puedes hacer que se vean de forma diferente para cada uno de los tamaños.
- Los contenedores centran y asignan espacios vacíos (padding) alrededor del contenido. Usa la clase container para fijar anchos adaptables con píxeles y container-fluid para anchos del 100%
- Las filas son envoltorios (wrappers) para las columnas. Cada columna tiene un padding horizontal (gutter) para controlar el espacio entre ellas. Las filas soportan clases que aplican un tamaño uniforme para las columnas y "gutter" para cambiar el espaciado.
- Las columnas son increíblemente flexibles. Hay 12 columnas disponibles por fila (row), permitiendo combinaciones de elementos que ocupen un número variable de columnas. La clase de las columnas indica el "ancho", el número de columnas que ocupan del total de 12. Por ejemplo, col-4 indica que ocupa 4/12 del ancho.

Grid: filas y columnas (bootstrap02.html)

Mismo ancho, ancho determinado por la clase y ancho ajustado al contenido...

```
<div class="container">
  <div class="row">
    <div class="col">
      Column
    </div>
  </div>
  <div class="col">
    Column
  </div>
  <div class="col">
    Column
  </div>
</div>
```

```
<div class="container">
  <div class="row">
    <div class="col">
      1 of 2
    </div>
    <div class="col">
      2 of 2
    </div>
  </div>
  <div class="row">
    <div class="col">
      1 of 3
    </div>
    <div class="col">
      2 of 3
    </div>
    <div class="col">
      3 of 3
    </div>
  </div>
</div>
```

```
<div class="container">
  <div class="row">
    <div class="col">
      1 of 3
    </div>
    <div class="col-6">
      2 of 3 (wider)
    </div>
    <div class="col">
      3 of 3
    </div>
  </div>
  <div class="row">
    <div class="col">
      1 of 3
    </div>
    <div class="col-5">
      2 of 3 (wider)
    </div>
    <div class="col">
      3 of 3
    </div>
  </div>
</div>
```

```
<div class="container">
  <div class="row
justify-content-md-center">
    <div class="col col-lg-2">
      1 of 3
    </div>
    <div class="col-md-auto">
      Variable width content
    </div>
    <div class="col col-lg-2">
      3 of 3
    </div>
  </div>
  <div class="row">
    <div class="col">
      1 of 3
    </div>
    <div class="col-md-auto">
      Variable width content
    </div>
    <div class="col col-lg-2">
      3 of 3
    </div>
  </div>
</div>
```


Fijar columnas desde el row

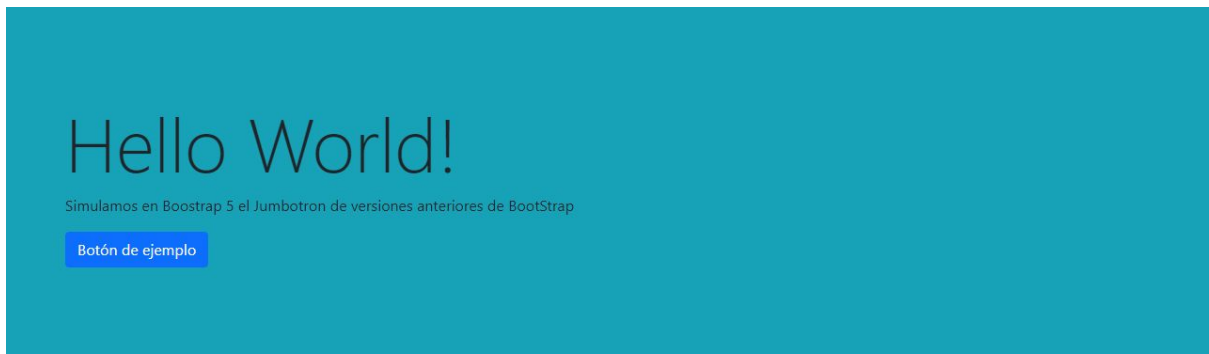
Usa la clase row-cols-(número) para fijar rápidamente el número de columnas apropiado para mostrar en una fila. Compruébalo cambiando el ancho de pantalla. Con auto, se ajusta al contenido

```
<div class="container">
  <div class="row row-cols-2">
    <div class="col">Column</div>
    <div class="col">Column</div>
    <div class="col">Column</div>
    <div class="col">Column</div>
  </div>
</div>
```

```
<div class="container">
  <div class="row row-cols-3">
    <div class="col">Column</div>
    <div class="col">Column</div>
    <div class="col">Column</div>
    <div class="col">Column</div>
  </div>
</div>
```

```
<div class="container">
  <div class="row row-cols-auto">
    <div class="col">Column</div>
    <div class="col">Column</div>
    <div class="col">Column</div>
    <div class="col">Column</div>
  </div>
</div>
```


Práctica. Intenta repetir el diseño mostrado



Primera

Lorem Ipsum is simply dummy text of the printing and typesetting industry. Lorem Ipsum has been the

Jarl

Primera B

Donec vel pellentesque sapien. Duis molestie ex sed nisl maximus pretium. Cras ullamcorper placerat massa, ut dapibus diam venenatis sed. Cras rutrum nisi urna, quis tempus eros luctus eu.

Segunda

Lorem Ipsum is simply dummy text of the printing and typesetting industry. Lorem Ipsum has been the industry's standard dummy text ever since the 1500s, when an unknown printer took a galley of type and scrambled it to make a type specimen book. It has survived not only five centuries, but also the leap into electronic typesetting, remaining essentially unchanged. It was popularised in the 1960s with the release of Letraset sheets containing Lorem Ipsum passages, and more recently with desktop publishing software like Aldus PageMaker including versions of Lorem Ipsum.

Jarl

Tercera

Lorem Ipsum is simply dummy text of the printing and typesetting industry. Lorem Ipsum has been the industry's standard dummy text ever since the 1500s, when an unknown printer took a galley of type and scrambled it to make a type specimen book. It has survived not only five centuries, but also the leap into electronic typesetting, remaining essentially unchanged. It was popularised in the 1960s with the release of Letraset sheets containing Lorem Ipsum passages, and more recently with desktop publishing software like Aldus PageMaker including versions of Lorem Ipsum.

Jarl

Anidando (Nesting)

Dentro de una columna podemos definir una nueva fila (row) y su conjunto correspondiente de columnas. Este conjunto de columnas anidadas también puede sumar hasta 12 de ancho, aunque no es obligatorio usar las 12.

```
<div class="container">
  <div class="row">
    <div class="col-sm-3">
      Level 1: .col-sm-3
    </div>
    <div class="col-sm-9">
      <div class="row">
        <div class="col-8 col-sm-6">
          Level 2: .col-8 .col-sm-6
        </div>
        <div class="col-4 col-sm-6">
          Level 2: .col-4 .col-sm-6
        </div>
      </div>
    </div>
  </div>
</div>
```

Columnas

- Como ya hemos visto, usando la arquitectura del Flexbox tenemos opciones para cambiar el ancho individual de las columnas y también modificar el ancho de un conjunto de columnas a través del row.
- Cuando trabajamos con la rejilla (grid) todo el contenido va dentro de columnas. La jerarquía de Bootstrap es: **Contenedores (container) > Filas (row) > Columnas (col)**
- Bootstrap incluye clases para crear diseños adaptables (responsive) rápidamente usando las 12 columnas y los 6 breakpoints (sm, md, ...)

Alineación vertical de columnas

Bootstrap permite alinear verticalmente las columnas.

```
<div class="row align-items-start">  
  <div class="col">  
    One of three columns  
  </div>  
  <div class="col">  
    One of three columns  
  </div>  
  <div class="col">  
    One of three columns  
  </div>  
</div>
```

```
<div class="row align-items-center">  
  <div class="col">  
    One of three columns  
  </div>  
  <div class="col">  
    One of three columns  
  </div>  
  <div class="col">  
    One of three columns  
  </div>  
</div>
```

```
<div class="row align-items-end">  
  <div class="col">  
    One of three columns  
  </div>  
  <div class="col">  
    One of three columns  
  </div>  
  <div class="col">  
    One of three columns  
  </div>  
</div>
```

<https://getbootstrap.com/docs/5.1/layout/columns/#vertical-alignment>

Alineación horizontal de columnas

En la versión 5 de Bootstrap se puede trabajar con idiomas que se escriben de derecha a izquierda (hebreo, árabe,...) por lo que para alinear a la derecha en idiomas occidentales se usa "start" y "end" para la izquierda.

```
<div class="row justify-content-start">  
  <div class="col-4">  
    One of two columns  
  </div>  
  <div class="col-4">  
    One of two columns  
  </div>  
</div>
```

```
<div class="row justify-content-center">  
  <div class="col-4">  
    One of two columns  
  </div>  
  <div class="col-4">  
    One of two columns  
  </div>  
</div>
```

```
<div class="row justify-content-end">  
  <div class="col-4">  
    One of two columns  
  </div>  
  <div class="col-4">  
    One of two columns  
  </div>  
</div>
```

<https://getbootstrap.com/docs/5.1/layout/columns/#horizontal-alignment>

Salto de columnas

El salto de columnas en Bootstrap requiere un pequeño truco (hacks) que consiste en incluir un elemento de ancho 100%

```
<div class="container">
  <div class="row">
    <div class="col-6 col-sm-3">.col-6 .col-sm-3</div>
    <div class="col-6 col-sm-3">.col-6 .col-sm-3</div>

    <!-- Force next columns to break to new line -->
    <div class="w-100"></div>

    <div class="col-6 col-sm-3">.col-6 .col-sm-3</div>
    <div class="col-6 col-sm-3">.col-6 .col-sm-3</div>
  </div>
</div>
```

<https://getbootstrap.com/docs/5.1/layout/columns/#column-breaks>

Reordenar columnas

Podemos cambiar el orden en el que se muestran las columnas, haciendo por ejemplo que sea la primera o la última, pudiendo también usar un número para la posición.

```
<div class="container">  
  <div class="row">  
    <div class="col order-last">  
      First in DOM, ordered last  
    </div>  
    <div class="col">  
      Second in DOM, unordered  
    </div>  
    <div class="col order-first">  
      Third in DOM, ordered first  
    </div>  
  </div>  
</div>
```

Desplazamiento de columnas (offset)

Puedes desplazar columnas de dos maneras, con la clase offset (por ejemplo **offset-md-4**) de forma adaptable o con margin.

```
<div class="container">
  <div class="row">
    <div class="col-md-4">.col-md-4</div>
    <div class="col-md-4 offset-md-4">.col-md-4 .offset-md-4</div>
  </div>
  <div class="row">
    <div class="col-md-3 offset-md-3">.col-md-3 .offset-md-3</div>
    <div class="col-md-3 offset-md-3">.col-md-3 .offset-md-3</div>
  </div>
  <div class="row">
    <div class="col-md-6 offset-md-3">.col-md-6 .offset-md-3</div>
  </div>
</div>
```

Práctica: Una web que se vea así

Verde				
Rojo	azul			
	amarillo		amarillo	
	azul claro	azul claro	azul claro	azul claro
	azul claro	azul claro	azul claro	azul claro
verde				

Gutters

Se trata del sistema de espaciado entre columnas usado para repartir el espacio de forma adaptable (responsive) y alinear el contenido.

Funcionamiento:

- Son los espacios entre el contenido de las columnas, son creados por padding horizontal.
- Comienzan en 24px de ancho.
- Pueden ajustarse responsivamente. Podemos usar breakpoints para modificar cuando aparecen estos paddings.

Gutters horizontales

Para el control del ancho de los gutters se utiliza la clase `.gx-*`. El contenedor debe ser ajustado en algunas ocasiones para evitar desbordamientos indeseados.

```
<div class="container px-4">
  <div class="row gx-5">
    <div class="col">
      <div class="p-3 border bg-light">Custom column
padding</div>
    </div>
    <div class="col">
      <div class="p-3 border bg-light">Custom column
padding</div>
    </div>
  </div>
</div>
```

Gutter Verticales

Para la anchura de los gutters verticales se usa la clase `gy`. En el caso de que ocurra un desbordamiento causado por esto usamos la clase `overflow-hidden` en el container para paliar este efecto.

```
<div class="container overflow-hidden">
  <div class="row gy-5">
    <div class="col-6">
      <div class="p-3 border bg-light">Custom column padding</div>
    </div>
    <div class="col-6">
      <div class="p-3 border bg-light">Custom column padding</div>
    </div>
    <div class="col-6">
      <div class="p-3 border bg-light">Custom column padding</div>
    </div>
    <div class="col-6">
      <div class="p-3 border bg-light">Custom column padding</div>
    </div>
  </div>
</div>
```

Gutters horizontal y vertical

La clase .g- se usa para controlar la anchura horizontal de los gutters.

```
<div class="container">
  <div class="row g-2">
    <div class="col-6">
      <div class="p-3 border bg-light">Custom column padding</div>
    </div>
    <div class="col-6">
      <div class="p-3 border bg-light">Custom column padding</div>
    </div>
    <div class="col-6">
      <div class="p-3 border bg-light">Custom column padding</div>
    </div>
    <div class="col-6">
      <div class="p-3 border bg-light">Custom column padding</div>
    </div>
  </div>
</div>
```

Sin gutters

Podemos eliminar los espacios predefinidos en el grid con g-0. Esto elimina los márgenes negativos y el padding horizontal de las columnas hijas.

```
<div class="row g-0">  
  <div class="col-sm-6 col-md-8">.col-sm-6 .col-md-8</div>  
  <div class="col-6 col-md-4">.col-6 .col-md-4</div>  
</div>
```